6장 라즈베리파이로 웹서버 만들기

강의 목표

1. 웹서버, 웹 응용 서버 및 웹 서버 프레임워크를 이해한다.
2. 플라스크를 설치할 수 있으며 기본적인 동작을 이해한다.
3. URL과 라우팅 테이블을 이해하고 활용할 수 있다.
4. URL 일부를 variable로 사용할 수 있다.
5. 템플릿 HTML과 파이썬 스크립트를 이용하여 동적 컨텐츠를 만들 수 있다.
6. 플라스크에서 GET과 POST를 이용하여 데이터를 수신할 수 있다.

들어가기 전에

- 복잡한 서버-클라이언트 시스템 이해
  - 대부분 처음으로 서버-클라이언트 시스템을 구축하기에 이 구조에 대해서 확실한 이해 필요

- 예제와 실습은 myenv 가상 환경에서 실행
  - 파이썬 코드 실행 전 반드시 가상 환경 활성화

```
pi@pi:~ $ source myenv/bin/activate
(myenv) pi@pi:~ $
```

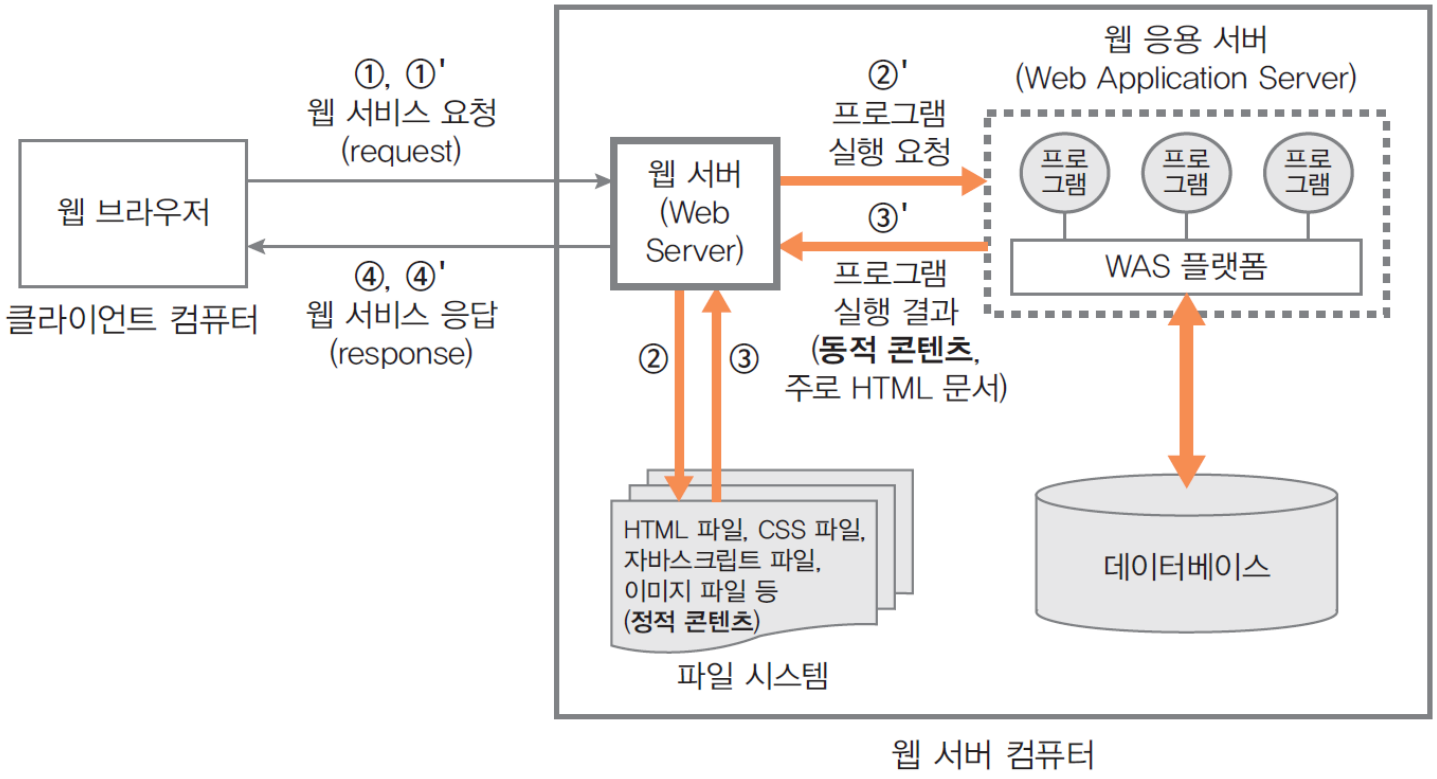
- 예제 파이썬 코드는 ch06 디렉터리에 저장하고 실행

```
(myenv) pi@pi:~ $ mkdir ch06
(myenv) pi@pi:~ $ cd ch06
(myenv) pi@pi:~/ch06 $
```

6.1 웹 서비스

웹 서버와 웹 응용 서버

- 웹서버
  - 웹 브라우저로부터 웹 문서에 대한 요청을 받고, 웹 브라우저로 웹 문서를 전송해주는 소프트웨어



웹 서버와 웹 응용 서버

- 웹 서버
  - 정적 콘텐츠
    - 미리 만들어진 콘텐츠. html 파일, CSS 파일, 자바스크립트 파일, 이미지 파일 등
  - 동적 콘텐츠
    - 웹 브라우저로부터의 요청에 따라 가변적으로 만들어지는 콘텐츠 (HTML 텍스트, 이미지 등)
- 웹 응용 서버
  - 동적 콘텐츠를 생성하는 서버
  - Tomcat, JBoss, Jeus, Web Sphere 등
  - 동적 콘텐츠의 사례
    - 학점이 4.0을 넘는 2학년 학생들의 명단 요청
    - 지난 10년 동안 가장 많이 팔린 자동차
    - 사진 이미지를 보내고 이 이미지에 담긴 사람들의 이름

웹 서버 사용 현황

- W3Techs Web Server Market Report 제공 (http://w3techs.com/technologies/overview/web_server)

제품	제조업체	사용 비율
Nginx	F5, Inc	32.3%
Cloudflare Server	Cloudflare. Inc.	28.1%
Apache	Apache Software Foundation	23.3%
LiteSpeed	LiteSpeed Technologies	15.2%

웹 서버 프레임워크

- 현대 웹서비스의 특징
  - 동적 컨텐츠가 대부분임
  - 웹 서버와 웹 응용 서버는 함께 존재

- 웹 서버 프레임워크
  - 웹 서버와 웹 응용 서버 플랫폼을 합친 소프트웨어
  - 대표적인 웹 서버 프레임워크
    - Spring: 자바 기반 프레임워크
    - Express.js: 자바스크립트 기반 프레임워크
    - Django, Flask: 파이썬 기반 프레임워크

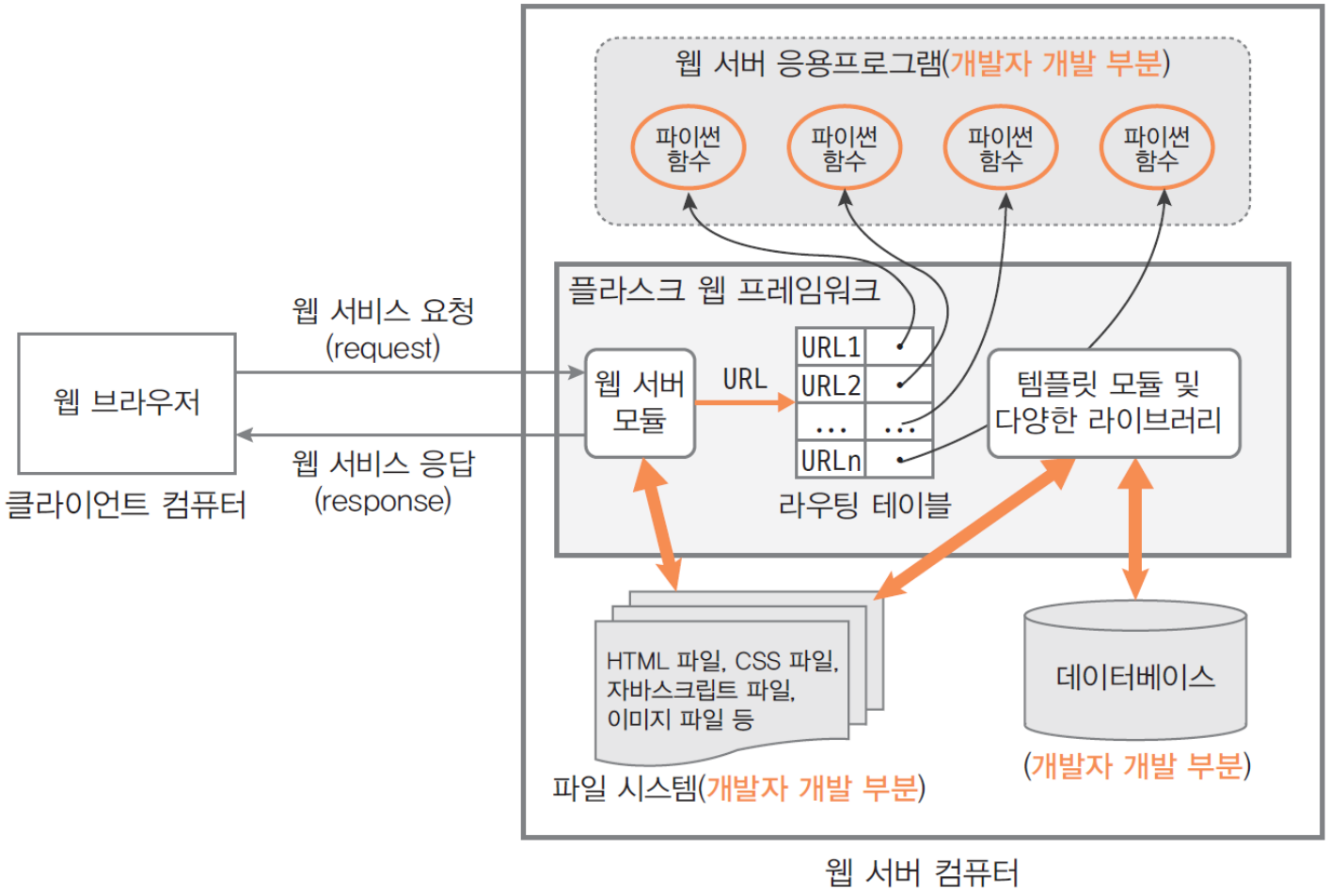
6.2 플라스크

플라스크 개요와 기본 동작

- 플라스크
  - 파이썬 언어로 작성된 웹 서버 프레임워크
  - 파이썬 언어로 웹 서버를 쉽게 구축하고 웹 서버 응용프로그램 작성을 쉽게 하는 다양한 파이썬 라이브러리와 모듈을 제공
  - 마이크로 웹 서버 프레임워크라고 부르기도 함
    - 이름에서 알 수 있듯이 소형 웹 서버를 구축하는데 매우 적합한 파이썬 전용 프레임워크
  - [URL, 함수]의 라우팅 테이블을 가지고 있음
    - 특정 URL에 대한 요청이 들어오면 대응하는 함수를 호출하여 동적 컨텐츠를 생성 및 전송

플라스크 개요와 기본 동작

- 플라스크 동작



플라스크 설치

- 플라스크 설치

```
(myenv) pi@pi:~ $ pip install flask
```

  - ~/myenv/lib/python3.13/site-packages에 라이브러리 설치

- 설치 여부 확인

```
(myenv) pi@pi:~ $ cd ./myenv/lib/python3.13/site-packages
(myenv) pi@pi:~/myenv/lib/python3.13/site-packages $ ls | grep flask
flask
flask-3.1.3.dist-info
(myenv) pi@pi:~/myenv/lib/python3.13/site-packages $
```

- 설치된 파이썬 모듈 확인 - flask 디렉터리 방문

```
(myenv) pi@pi:~/myenv/lib/python3.13/site-packages $ cd flask
(myenv) pi@pi:~/myenv/lib/python3.13/site-packages/flask $ ls
app.py          globals.py __pycache__ testing.py
blueprints.py helpers.py py.typed       typing.py
cli.py          __init__.py sansio      views.py
config.py       json        sessions.py wrappers.py
ctx.py          logging.py signals.py
debughelpers.py __main__.py templating.py
(myenv) pi@pi:~/myenv/lib/python3.13/site-packages/flask $
```

플라스크 활용 기본 사례 (1)

- 플라스크를 이용한 간단한 웹 서버
  - ~/ch06/hello_flask.py에 다음 파일 저장

```
from flask import Flask

app = Flask(__name__) # Flask 객체 생성

@app.route('/') # 다음 2라인이 라우팅 테이블에 ['/':home]를 저장하도록 함
def home():
    return '<h2>Hello, Flask</h2>'

if __name__ == '__main__': # 이 프로그램이 독립적으로 실행되는 경우
    app.run(host='0.0.0.0', port=8080, debug=True) # app.run() 함수 실행
```

- from flask import Flask 란?
  - 설치된 flask 모듈에서 Flask 클래스만 임포트하도록 지시
  - Flask 클래스 코드는 app.py(~/myenv/lib/python3.13/dist-packages/flask/app.py)에 작성되어 있음

플라스크 활용 기본 사례 (2)

- app = Flask(__name__)
  - Flask 객체를 생성하고 app라고 명명함
- @app.route('/')
  - @app.route('/')는 파이썬에서 데코레이터(decorator)라고 부름
  - '/' URL과 그 다음 라인에 선언된 함수를 한 쌍으로 하여 라우팅 테이블에 저장하도록 지시
    - 라우팅 테이블에는 ['/': home] 항목이 저장
    - ['/': home] 쌍이라는 것을 알리기 위해 @app.route('/')와 다음 라인의 def home(): 사이에는 주석과 공백 외 어떤 것도 들어가면 안됨
- def home():
  - '/' URL로 접속할 때 실행되는 함수
  - 단순히 '<h2>Hello, Flask</h2>'을 리턴

플라스크 활용 기본 사례 (3)

- if __name__ == '__main__':
  - hello_flask.py이 독립적으로 실행되고 있다면 __name__ 변수에 '__main__'이 저장된 채 실행
  - 현재, 독립적으로 실행되므로 if문은 True
- app.run(host='0.0.0.0', port=8080, debug=True)
  - app는 Flask 클래스의 객체이므로 Flask 클래스의 run() 함수가 실행됨
  - app.run() 함수에는 host, port, debug의 매개변수가 있음

플라스크 활용 기본 사례 (4)

- app.run(host='0.0.0.0', port=8080, debug=True)
  - host 매개변수
    - app.run()이 웹 브라우저로부터 연결을 받아 줄 IP 주소 지정
    - 서버가 여러 개의 LAN에 연결된 경우, 서버는 여러 개의 IP 가짐
      - 특별한 IP로만 연결하기 위해서는, host='192.168.0.5'로 IP를 명시적으로 지정
      - 여러 IP중 아무것이나 연결하기 위해서는, host='0.0.0.0'
      - 서버 자체내에서만 연결하기 위해서는, host='127.0.0.1'
  - port 매개변수
    - app.run()이 웹 브라우저로부터 접속을 기다리는 TCP/IP 포트 번호를 지정
    - 여기서는 8080 포트에서 기다림
      - 디폴트는 5000

플라스크 활용 기본 사례 (5)

- app.run(host='0.0.0.0', port=8080, debug=True)
  - debug 매개변수
    - 웹 서버 응용프로그램을 개발을 돕는 기능
    - debug=True
      - *app.run()*은 실행 과정을 계속 출력하여 진행되고 있는 작업 내용들을 보여줌
      - 개발자는 이 출력을 보고, 웹 브라우저가 어떤 URL로 접속해 왔는지, 어떤 웹 콘텐트를 전송하였는지 등을 확인하면서, 프로그램의 문제를 발견하거나 디버깅할수 있음
    - 예) hello_flask.py 프로그램 실행 도중, 개발자가 hello_flask.py 소스 코드를 수정하면 재실행시키지 않아도 수정된 hello_flask.py를 자동으로 적재하고 재실행
      - 반대로 *False*이면 반드시 재실행하여야 함

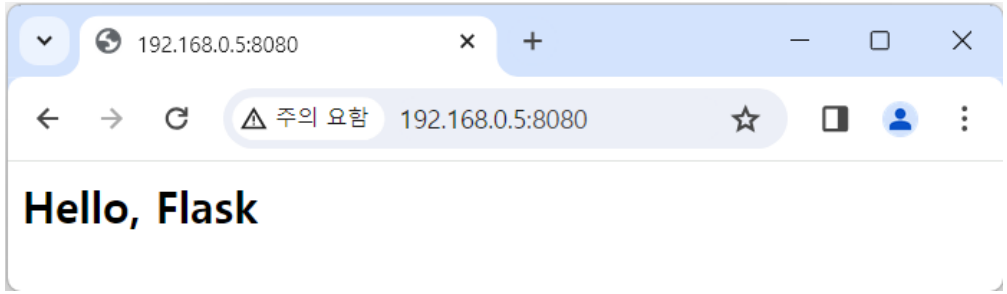
플라스크 활용 기본 사례 (6)

- hello_flask.py 프로그램 실행

```
(myenv) pi@pi:~/ $ cd ~/ch06
(myenv) pi@pi:~/ch06 $ python hello_flask.py
* Serving Flask app "hello_flask" (lazy loading)
* Environment: production
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Debug mode: on
* Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger PIN: 315-171-481
[지금 app.run( )에서 웹 브라우저의 접속을 기다리고 있는 상태]
```

플라스크 활용 기본 사례 (7)

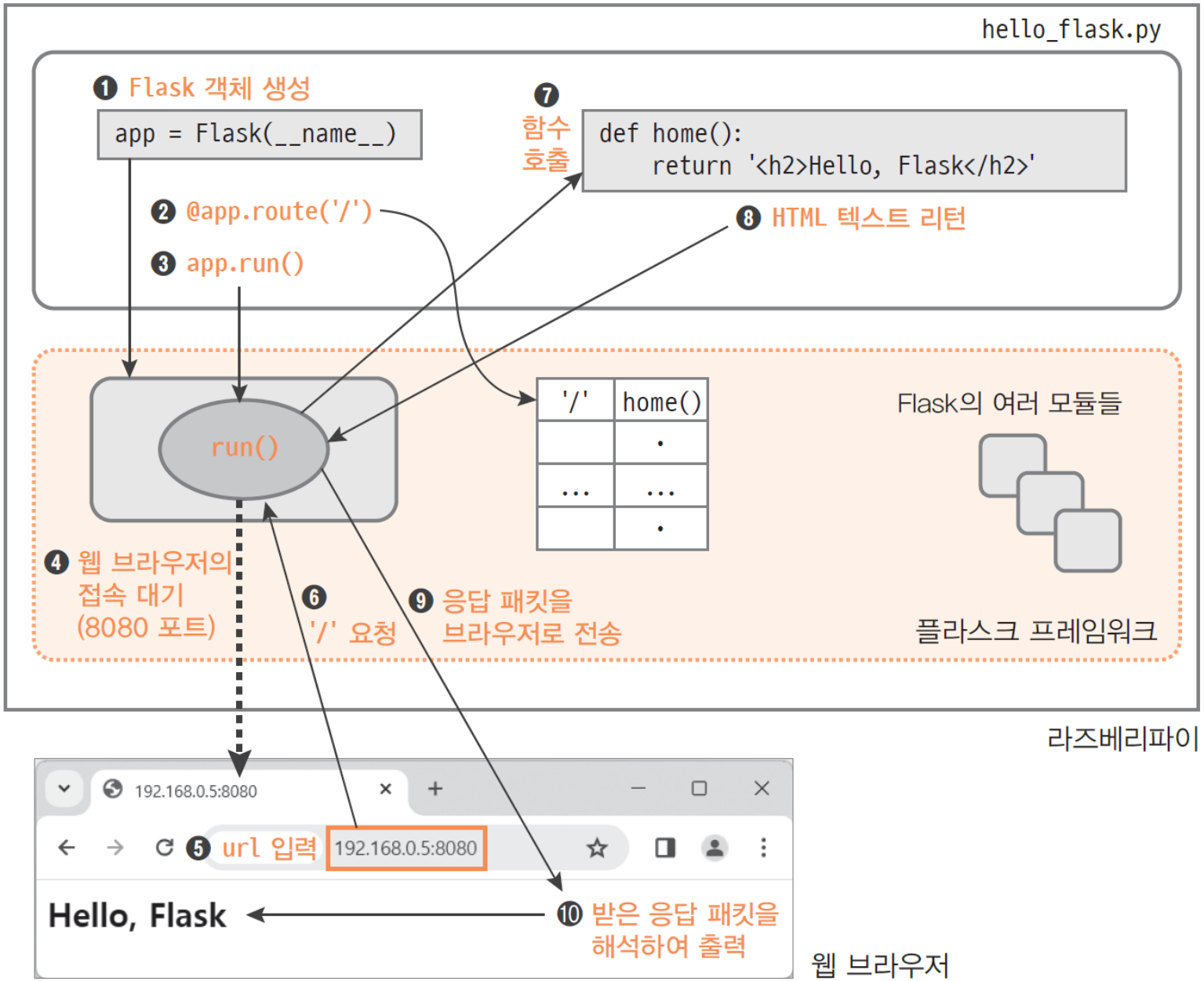
- 웹 브라우저로 웹 서버에 접속



- hello_flask.py의 출력

```
(myenv) pi@pi:~/ch06 $ python hello_flask.py
* Serving Flask app "hello_flask" (lazy loading)
...(생략)
* Debugger PIN: 315-171-481
192.168.0.50 - - [09/May/2023 13:53:23] "GET / HTTP/1.1" 200 -
192.168.0.50 - - [09/May/2023 13:53:23] "GET /favicon.ico HTTP/1.1" 404 -
```

```
(myenv) pi@pi:~/ch06 $ python hello_flask.py
```



예제 6-1 플라스크 웹 서버 만들기 연습

웹 브라우저에서 '192.168.0.5:8080/hello/'라는 URL을 입력하면 다음과 같은 웹 페이지가 출력되도록, 플라스크를 이용하여 웹 서버를 구축하라.

6-1.py

```
from flask import Flask

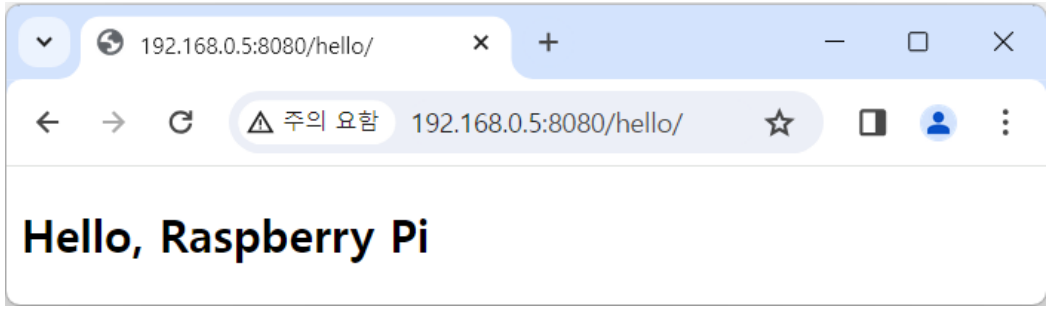
app = Flask(__name__)

@app.route('/hello/')
def hello( ):
    return """
        <!DOCTYPE html>
        <html>
            <body>
                <h2>Hello, Raspberry Pi</h2>
            </body>
        </html>
        """

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

예제 6-1 (계속)


```
(myenv) pi@pi:~/ch06 $ python 6-1.py
* Serving Flask app "6-1" (lazy loading)
....(생략)
* Debugger PIN: 235-625-251
192.168.0.50 - - [13/May/2023 08:46:01] "GET /hello/ HTTP/1.1" 200 -
```



6.3 URL 라우팅

URL과 라우팅 테이블 리뷰

- 동적 컨텐츠의 요청
  - URL의 요청이 동적 콘텐츠에 대한 요청인 경우
    - 웹 서버는 동적 콘텐츠를 만들어내는 작업을 수행
    - 플라스크는 웹 서버에서 요청 URL에 대해 동적 콘텐츠를 생성하는 함수가 실행되도록, URL 라우팅 테이블이라고 불리는 곳에 [URL: 함수]의 형태로 저장

```
@app.route('/')    # 라우팅 테이블에 ['/':home] 항목 저장
def home():
   return '<h2>Hello, Flask</h2>'
```

예제 6-2 URL 라우팅으로 동적 콘텐츠를 만들어 응답하기

웹 브라우저에서 '/kitae/ '의 URL로 요청하는 경우와 '/jmlee/ '의 URL로 요청하는 경우, 각각 다음과 같이 출력되도록, 플라스크를 활용하여 웹 서버 프로그램을 작성하라.

6-2.py

```
from flask import Flask

app = Flask(__name__)

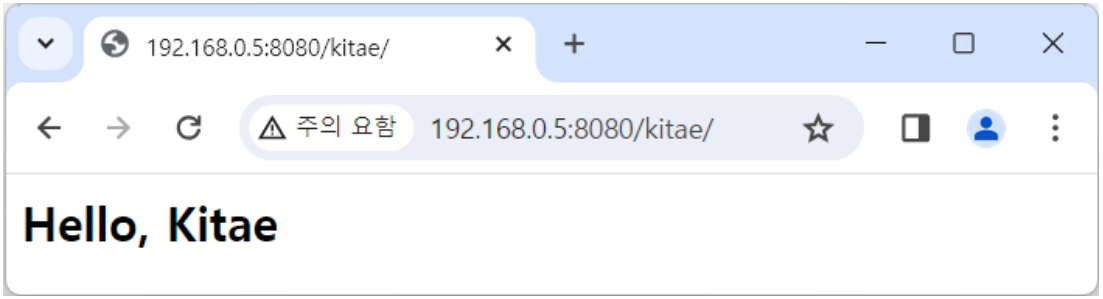
@app.route('/kitae/')
def doKitae( ):
    return "<h2>Hello, Kitae</h2>"

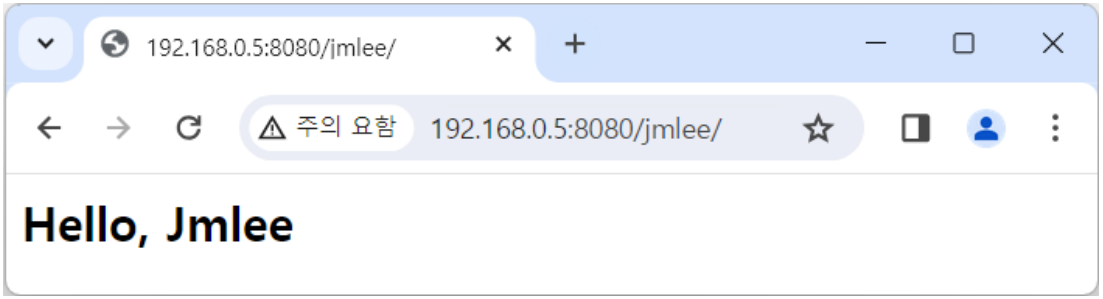
@app.route('/jmlee/')
def doJmlee( ):
    return "<h2>Hello, Jmlee</h2>"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

예제 6-2 (계속)


```
(myenv) pi@pi:~/ch06 $ python 6-2.py
* Serving Flask app "6-2" (lazy loading)
....(생략)
* Debugger PIN: 235-625-251
192.168.0.50 - - [13/May/2023 08:55:19] "GET /kitae/ HTTP/1.1" 200 -
192.168.0.50 - - [13/May/2023 08:56:21] "GET /jmlee/ HTTP/1.1" 200 -
```





URL 끝에 /가 있는 경우와 없는 경우 차이

- 다음 2 URL은 서로 다른 것으로 다룸
  - /kitae
  - /kitae/

웹 서버 코드	웹 브라우저 입력	플라스크 처리
@app.route('/kitae') def home(): ...	IP:8080/kitae	정상 처리 : home() 함수 호출
	IP:8080/kitae/	오류 처리 : 요청을 발견할 수 없음
@app.route('/kitae/') def home(): ...	IP:8080/kitae/	정상 처리 : home() 함수 호출
	IP:8080/kitae	정상 처리 : 이 경우 브라우저는 IP:8080/kitae로 요청을 하지만, 플라스크가 IP:8080/kitae를 라우팅 테이블에서 찾지 못하고, IP:8080/kitae/가 있어 IP:8080/kitae/로 재요청하라는 308 응답 코드(Permanent Redirect)를 보낸다. 그러면 브라우저는 IP:8080/kitae/로 다시 요청하고, 서버는 home() 함수 호출

예제 6-3 URL 끝에 /가 있는 경우와 없는 경우 웹 서버 처리

예제 6-2의 코드에서 @app.route ('/kitae/ ')를 @app.route ('/kitae')로 수정하여, 예제 6-2에서 웹 브라우저 사용자가 입력한 URL 그대로 접속하면 그 결과가 어떻게 될 것인지 예측하여 보라.

6-3.py

```
from flask import Flask

app = Flask(__name__)

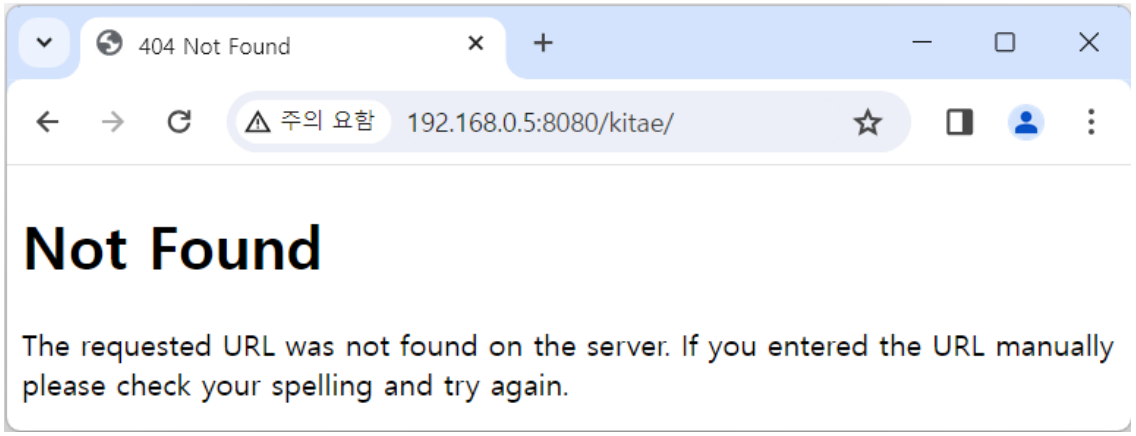
@app.route('/kitae') # URL 끝에 /가 없는 경우이다.
def doKitae( ):
    return "<h2>Hello, Kitae</h2>"

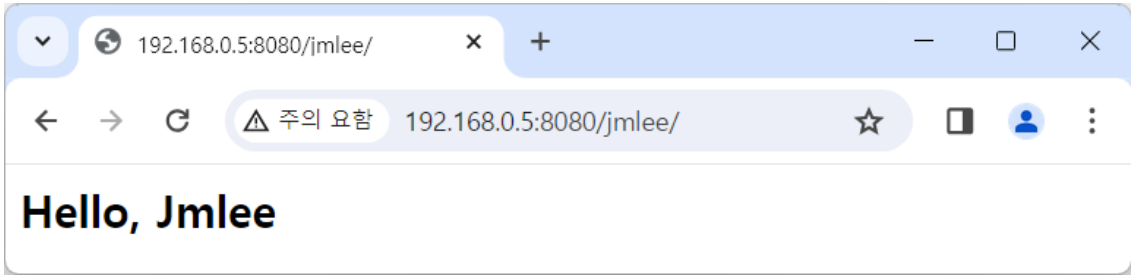
@app.route('/jmlee/') # URL 끝에 /가 있는 경우이다.
def doJmlee( ):
    return "<h2>Hello, Jmlee</h2>"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

예제 6-3 (계속)


```
(myenv) pi@pi:~/ch06 $ python 6-3.py
* Serving Flask app "6-3" (lazy loading)
....(생략)
* Debugger PIN: 235-625-251
192.168.0.50 - - [12/May/2023 12:43:13] "GET /kitae/ HTTP/1.1" 404 -
192.168.0.50 - - [12/May/2023 12:43:23] "GET /jmlee/ HTTP/1.1" 200 -
```





정적 콘텐츠 다루기

- 정적 콘텐츠의 종류
  - HTML 파일이나 CSS 파일, 자바스크립트 파일, 이미지 파일, 오디오 파일, 동영상 파일 등 파일 형태로 만들어져서 변환이 안 되는 형태의 콘텐츠
- 정적 콘텐츠를 저장해두는 디렉터리
  - 플라스크는 정적 콘텐츠를 저장해두는 디렉터리 명을 static으로 정해 놓음, 물론 별도의 설정을 통하여 변경 가능
- 정적 콘텐츠에 대한 라우팅 테이블 만들지 않음
  - 디렉터리가 정해져 있으므로 라우팅 테이블에 포함되지 않음
- 정적 콘텐츠에 대한 라우팅
  - 플라스크는 라우팅 테이블에 자동으로 ['/static/파일이름', static()]의 항목을 저장
  - '/static/파일이름'의 형식으로 요청된 URL에 대해서는 자동으로 static() 함수가 실행
  - 함수는 단순히 요청받은 파일을 읽어 웹 브라우저로 전송

예제 6-4. 정적 콘텐츠 중 이미지 처리 - /static/image.jpg

~/06/static 디렉터리에 image.jpg 파일이 있고 이를 웹 브라우저에서 IP:8080/static/image.jpg로 입력하면 image.jpg를 전송하는 웹 서버 프로그램을 작성하라. 이 예제는 웹 서버 개발자가 정적 콘텐츠를 저장해두고 웹 브라우저의 요청을 받아 정적 콘텐츠를 전송하는 사례이다. 웹 서버 프로그램에 정적 콘텐츠 처리를 위한 코드를 작성할 필요는 없다. 단지 다음과 같이 플라스크만 실행하면 된다.

* 이 예제를 실행시키기 전에 ~/ch06 디렉터리에
  static 디렉터리를 만들고 image.jpg 파일을 하나 저장해두어야 한다.
  5장에서 만든 이미지 파일을 저장하면 좋을 듯 하다.

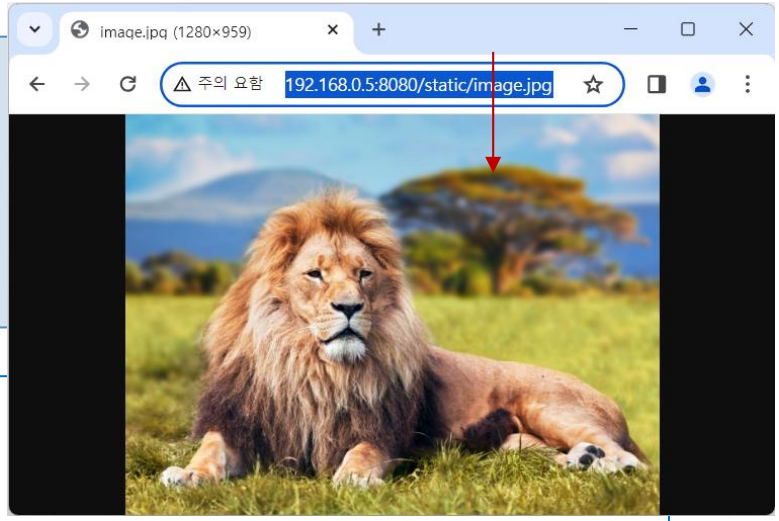
http://192.168.0.5:8080/static/image.jpg

6-4.py

```
from flask import Flask

app = Flask(__name__)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```




```
(myenv) pi@pi:~/ch06 $ python 6-4.py
* Serving Flask app '6-4'
...
* Debugger PIN: 119-627-469
192.168.0.50 - - [23/Dec/2023 14:02:59] "GET /static/image.jpg HTTP/1.1" 200 -
192.168.0.50 - - [23/Dec/2023 14:03:08] "GET /static/image.jpg HTTP/1.1" 304 -
192.168.0.50 - - [23/Dec/2023 14:03:16] "GET /static/image.jpg HTTP/1.1" 304 -
```

variable 사용 - URL의 일부를 함수의 매개 변수로 활용(1)

- variable 사용
  - URL 문자열의 일부를 함수의 매개 변수에 전달하여, 함수가 처리할 작업을 구체적으로 지시

```
@app.route('/.../<[converter:]variable_name>/.../')
def func(variable_name): # URL 문자열 내 variable_name 부분의 값이 매개변수에 전달
    ...
```

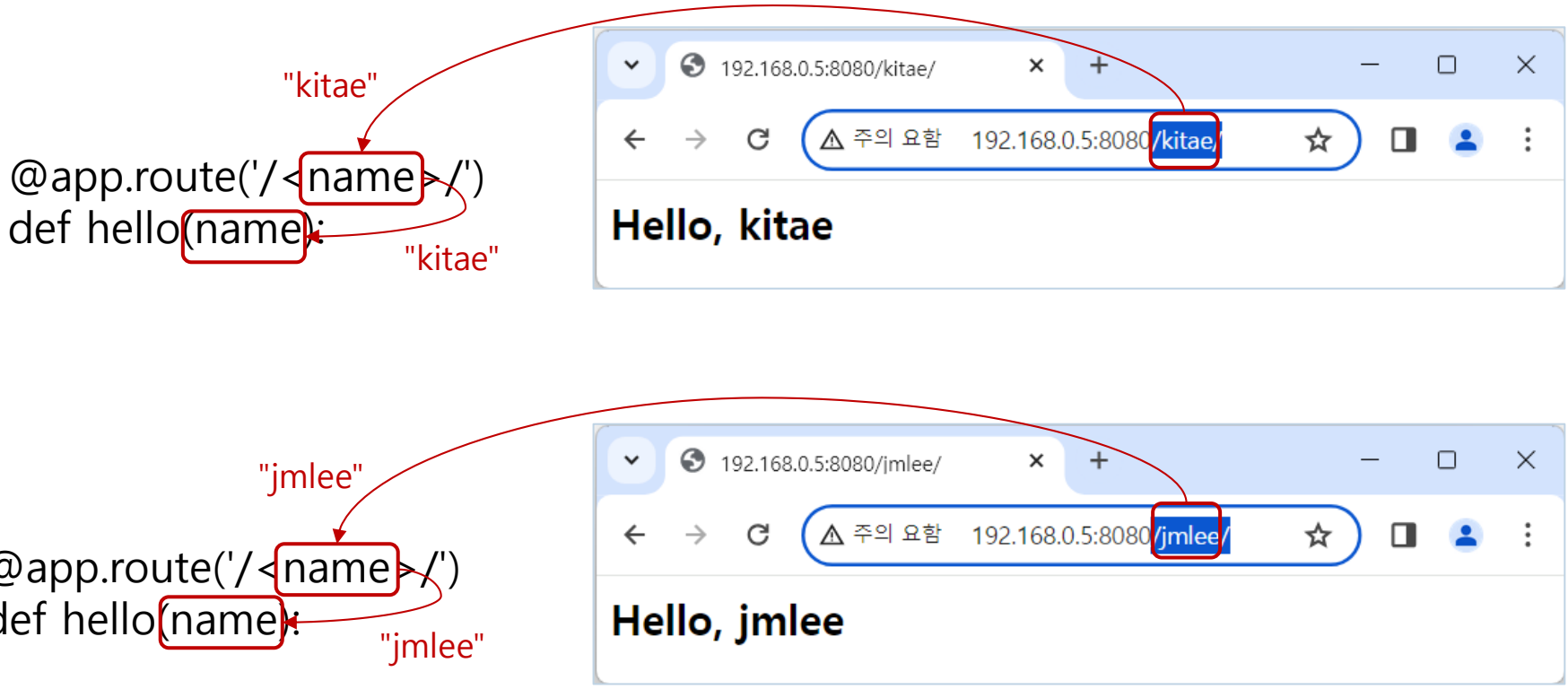
- [convert:]는 생략 가능

```
@app.route('/<name>/')
def hello(name):
```

192.168.0.5:8080/jmlee/ -> 문자열 "jmlee"을 hello() 함수의 매개변수에 전달
192.168.0.5:8080/kitae/ -> 문자열 "kitae"을 hello() 함수의 매개변수에 전달

variable 사용 - URL의 일부를 함수의 매개 변수로 활용(2)

- variable 사용



variable 사용 - URL의 일부를 함수의 매개 변수로 활용(3)

- convert
  - URL은 문자열
  - variable 기법의 의해 URL의 일부를 함수의 매개변수로 전달할 때, 함수에 따라서는 문자열 대신에 정수나 실수 등 다른 타입의 값으로 전달

```
@app.route('/.../<[int:]variable_name>/.../')
```

  - converter에 사용가능한 타입

타입	설명
int	정수로 변환
float	실수로 변환
path	경로명에 있는 '/'도 포함된 문자열. converter의 디폴트 타입
uuid	UUID 문자열로 변환

예제 6-5 variable을 사용하여 예제 6-4를 하나의 URL로 매핑

예제 6-3에서 @app.route ('/kitae/ ')와 @app.route ('/jmlee/ ') URL에 대하여 2 함수를 사용하였다. variable 기능을 사용하여 예제 6-3을 하나의 URL과 하나의 함수를 사용하도록 수정하라.

6-5.py

```
from flask import Flask

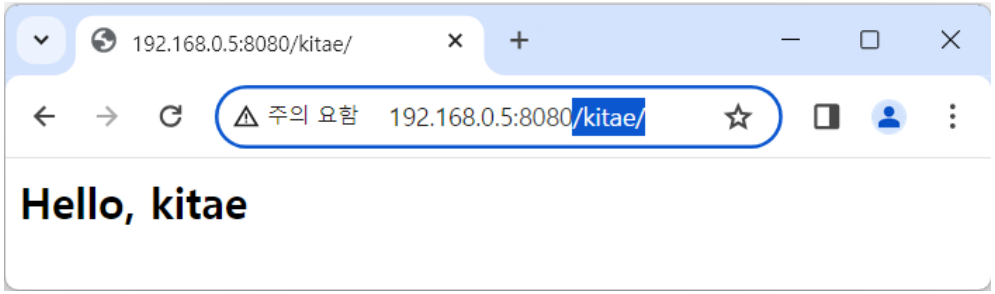
app = Flask(__name__)

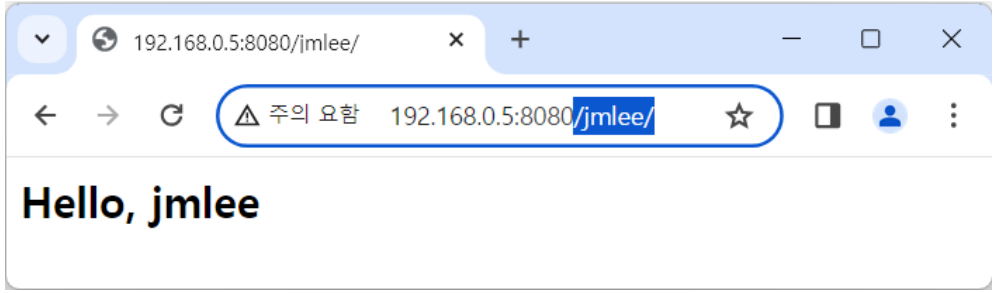
@app.route('/<name>/')
def hello(name):
    if name == 'kitae':
        str = "<h2>Hello, Kitae</h2>"
    elif name == 'jmlee':
        str = "<h2>Hello, Jmlee</h2>"
    else:
        str = "" # 빈 내용을 리턴한다.
    return str

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

예제 6-5 (계속)


```
(myenv) pi@pi:~/ch06 $ python 6-5.py
* Serving Flask app "6-5" (lazy loading)
....(생략)
* Debugger PIN: 235-625-251
192.168.0.50 - - [13/May/2023 09:12:43] "GET /kitae/ HTTP/1.1" 200 -
192.168.0.50 - - [13/May/2023 09:17:31] "GET /jmlee/ HTTP/1.1" 200 -
```





예제 6-6 variable 이용 실습 – 은행 계좌에 입금, 출금, 조회 가능한 웹 서버 구축

URL에 variable을 활용하여, 은행 계좌에 입금하고, 출금하고, 조회가 가능한 웹 서버를 간단히 구축하라. 초기 잔액은 0원으로 하고, 웹 브라우저에서는 다음 3가지 형태의 URL을 입력한다.

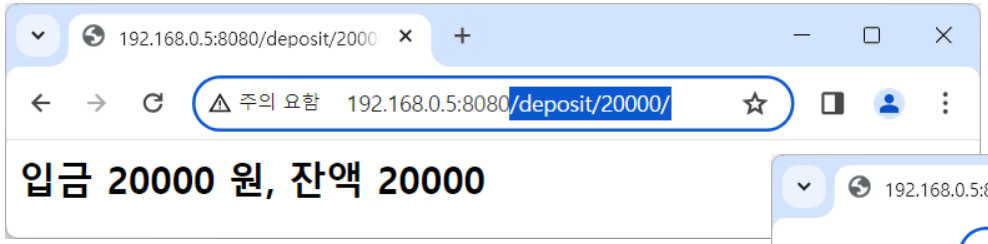
- http://192.168.0.5:8080/deposit/20000 - 20000원 입금
- http://192.168.0.5:8080/withdraw/13000 - 13000원 출금
- http://192.168.0.5:8080/inquiry/ - 잔액 조회

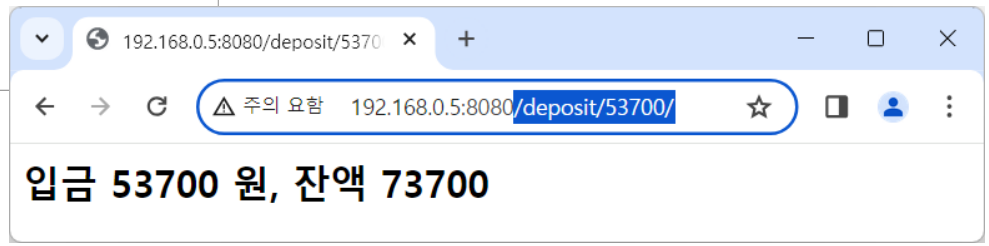
6-6.py

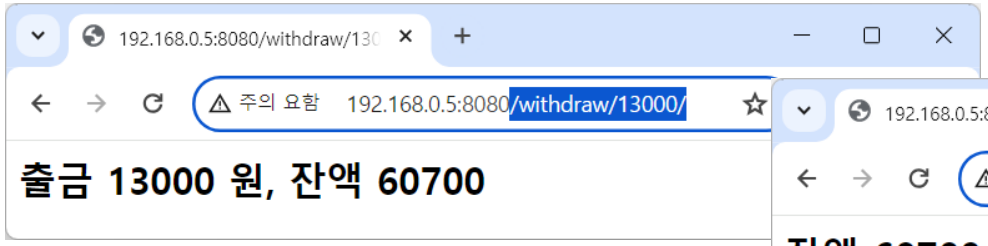
```
from flask import Flask
app = Flask(__name__)
balance = 0 # 잔액. 초기에는 0원
@app.route('/<op>/<int:money>/') # 2개의 variable: op와 money
def bank(op, money):
    global balance # 전역 변수 활용
    if op == 'deposit': # 입금
        balance += money # 잔액에 더하기
        return "<h2>입금 %d 원, 잔액 %d</h2>" % (money, balance)
    elif op == 'withdraw':
        # 인출금액이 잔액보다 많으면, 잔액만큼만 인출
        withdraw = balance if balance < money else money
        balance -= withdraw
        return "<h2>출금 %d 원, 잔액 %d</h2>" % (withdraw, balance)
    else:
        return "deposit 또는 withdraw 중 하나를 입력하세요"
@app.route('/inquiry/')
def inquiry( ):
    return "<h2>잔액 %d </h2>" % (balance)
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

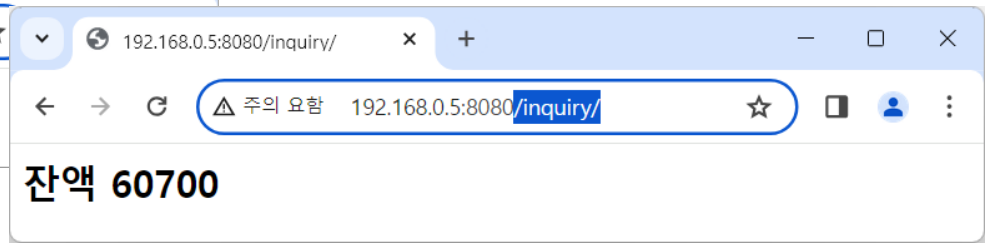
예제 6-6. variable을 이용 실습 – 은행 구좌에 저금, 출금, 조회 가능한 웹 서버 구축


```
(myenv) pi@pi:~/ch06 $ python 6-6.py
* Serving Flask app "6-6" (lazy loading)
....(생략)
* Debugger PIN: 235-625-251
192.168.0.50 - - [13/May/2023 00:22:17] "GET /deposit/20000/ HTTP/1.1" 200 -
192.168.0.50 - - [13/May/2023 00:22:55] "GET /deposit/53700/ HTTP/1.1" 200 -
192.168.0.50 - - [13/May/2023 00:23:31] "GET /withdraw/13000/ HTTP/1.1" 200 -
192.168.0.50 - - [13/May/2023 00:24:00] "GET /inquiry/ HTTP/1.1" 200 -
```





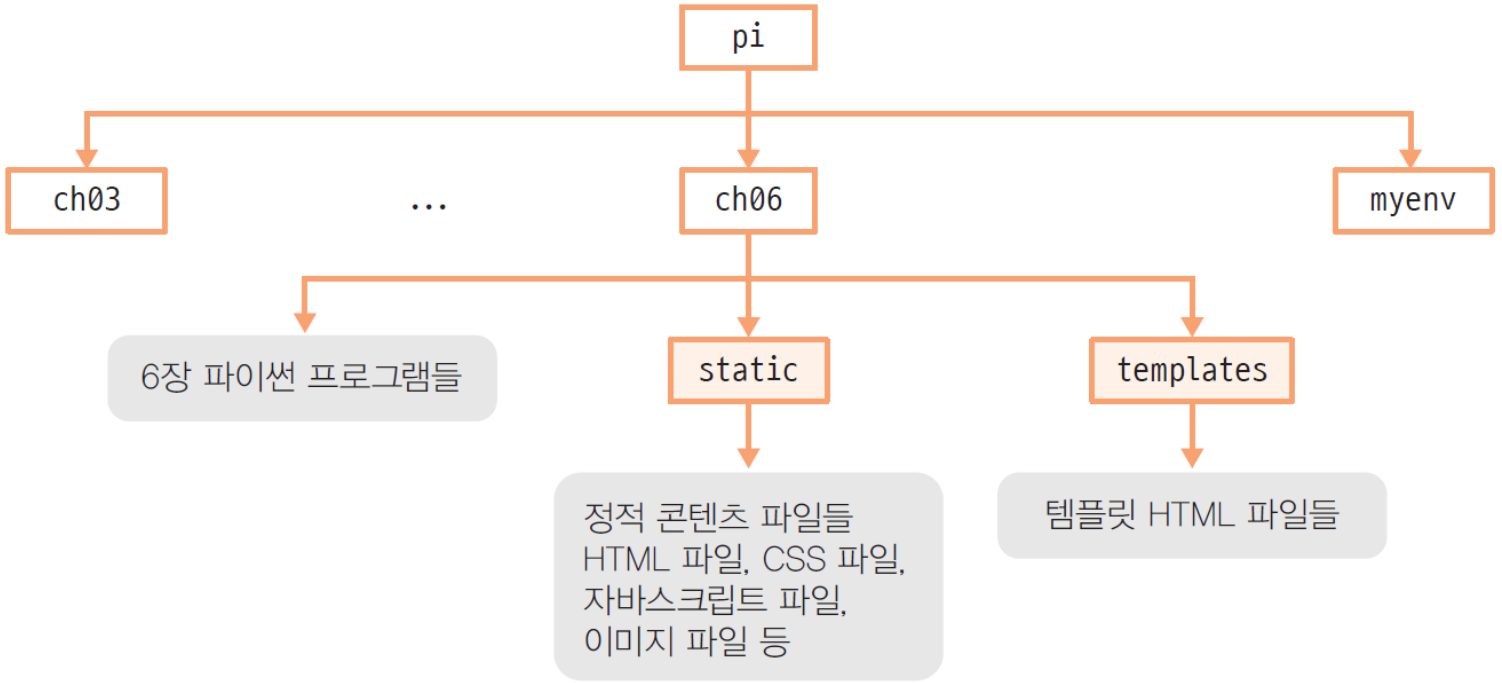




6.4템플릿 HTML 파일의 동적 변경

플라스크의 디렉터리 구조

- 2가지 디렉터리
  - static 디렉터리
    - 정적 콘텐츠를 두는 디렉터리
  - templates 디렉터리
    - 동적 콘텐츠를 생성하기 위해 템플릿을 두는 디렉터리



템플릿 HTML과 render-template() 함수

- 템플릿 HTML
  - 웹 서버를 구축하다 보면 많은 동적 콘텐츠들이 동일한 HTML 텍스트를 기반으로 조금씩 변경되는 형태가 대부분
  - 웹 브라우저로부터 모든 요청에 대해 HTML 문서를 동적으로 생성하는 것은 매우 힘든 작업
  - 플라스크는 웹 서버에서 HTML 문서를 동적으로 생성하기 쉽도록 템플릿 기능을 제공
    - 템플릿 HTML 파일을 작성하여 templates 디렉터리에 저장
    - render_template() 함수를 사용하여 템플릿 HTML 파일에 약간의 수정을 가해 동적 HTML 파일 생성 - render_template() 함수가 템플릿 HTML 파일을 수정하는 작업을 렌더링(rendering)이라고 부름

템플릿 HTML과 render-template() 함수

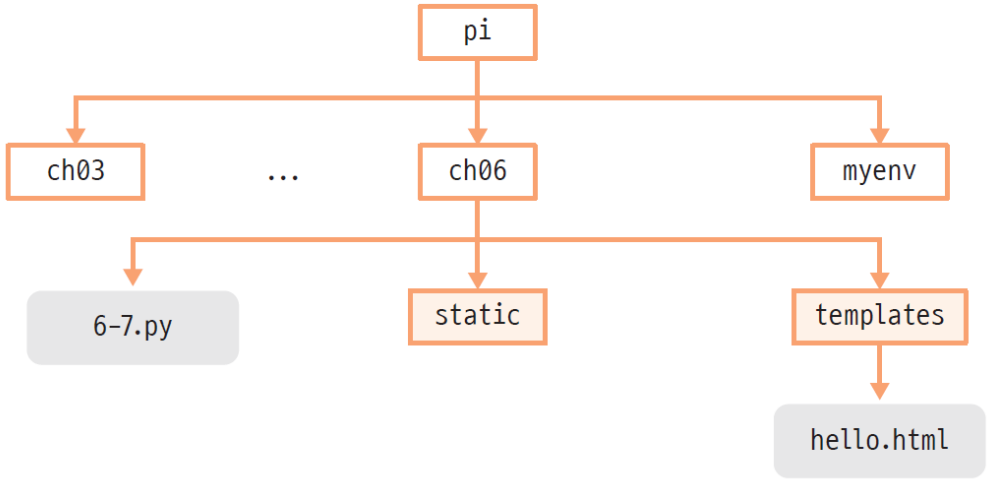
- render_template() 함수 활용
  - 템플릿 HTML 파일을 작성하고 render_template() 함수를 사용하여 동적으로 HTML 문서를 생성하는 방법

```
from flask import render_template
```

```
render_template('hello.html')
```

예제 6-7 템플릿 HTML 파일을 render_template() 함수로 수정 없이 그대로 전송

이 예제는 hello.html을 템플릿 HTML 파일로 작성하여 templates 디렉터리에 저장해놓고, 웹 브라우저의 요청 시에 hello.html을 수정 없이 그대로 전송하는 매우 간단한 사례이다. 디렉터리 구조는 다음과 같다.



hello.html

```
<!doctype html>
<html>
<head>
    <title>Template 사용하기</title>
</head>
<body>
    <h2>Template 사용하기</h2>
    <hr>
    <h2>Hello, Flask!</h2>
</body>
</html>
```

6-7.py

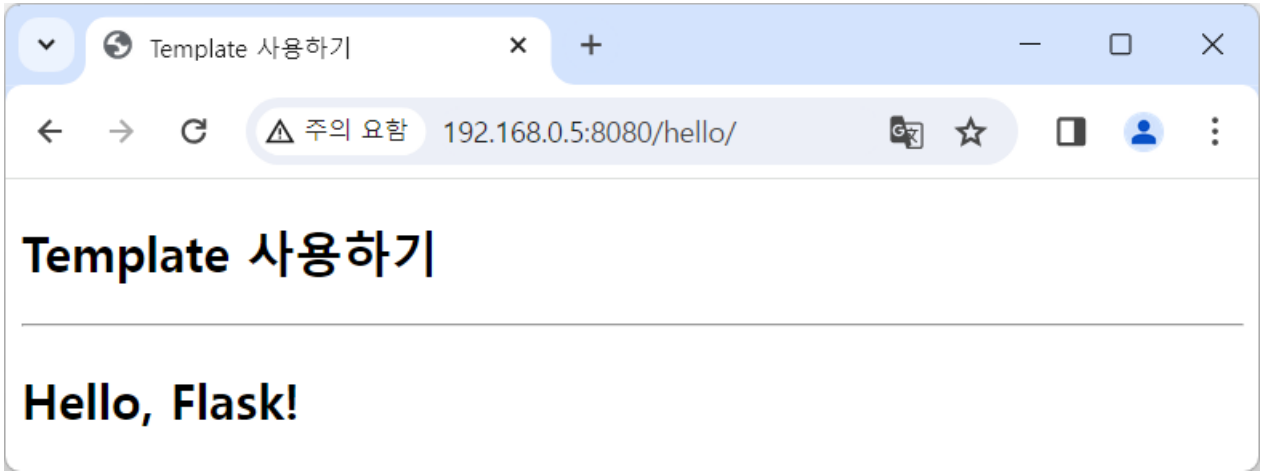
```
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/hello/')
def hello( ):
    return render_template('hello.html')

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

예제 6-7 템플릿 HTML 파일을 render_template() 함수로 수정 없이 그대로 전송


```
(myenv) pi@pi:~/ch06 $ python 6-7.py
* Serving Flask app "6-7" (lazy loading)
...(생략)
* Debugger PIN: 235-625-251
192.168.0.50 - - [15/May/2023 23:37:54] "GET /hello/ HTTP/1.1" 200 -
```



템플릿 HTML 파일에 스크립트 사용

- render_template() 함수
  - 템플릿 HTML 문서를 렌더링할 때, HTML 파일 내에 작성된 플라스크 스크립트(script) 코드를 실행하여 HTML 파일을 변형
- 템플릿 HTML 파일 내에 스크립트 삽입

{{변수 또는 수식}} # 변수나 수식을 계산한 값을 문자열로 출력
{% 스크립트 %} # 한 라인의 스크립트만 작성 가능
{% endif %} 와 {% endfor %} # if문이나 for문의 블록 종료

```
{% if(name == 'jmlee'): %}
    {% print('Jae Moon Lee!') %}
{% elif(name == 'kitae'): %}
    {% print('Ki Tae Hwang!') %}
{% else: %}
    {% print('Unknown') %}
{% endif %}
```

```
{% if(name == 'jmlee'): %}
{% print('Jae Moon Lee!') %}
{% elif(name == 'kitae'): %}
{% print('Ki Tae Hwang!') %}
{% else: %}
{% print('Unknown') %}
{% endif %}
```

템플릿 HTML 파일에 스크립트 사용

- 스크립트 코드의 변수에 값 전달

```
render_template(
   target,    # 템플릿 HTML 파일명
   name1 = value1,  # target 스크립트 코드의 변수에 값 전달
   name2 = value2, ,  # target 스크립트 코드의 변수에 값 전달
   ...
   nameN = valueN , target 스크립트 코드의 변수에 값 전달
)
리턴값: 템플릿 HTML 파일을 수정한 HTML 문서
```

(1) 리턴할 빈 문자열을 만든다.
(2) target HTML 파일을 라인 별로 읽는다.
(3) 읽은 라인에 파이썬 스크립트가 없으면, 라인을 리턴할 문자열에 연결한다.
(4) 만일, 라인에 파이썬 스크립트가 있으면, render_template()의 매개변수로 전달받은 변수와 동일한 이름의 변수가 스크립트 코드에 있으면 변수에 해당 값을 저장한다. 그리고 스크립트 코드를 실행한다. 이때 print() 문에 의한 실행 결과가 있으면 실행 결과를 리턴할 문자열에 연결한다.
(5) 리턴하기 위해 만들어진 문자열을 리턴한다.

템플릿 HTML 파일에 스크립트 사용

- 스크립트 코드의 변수에 값 전달
  - 예

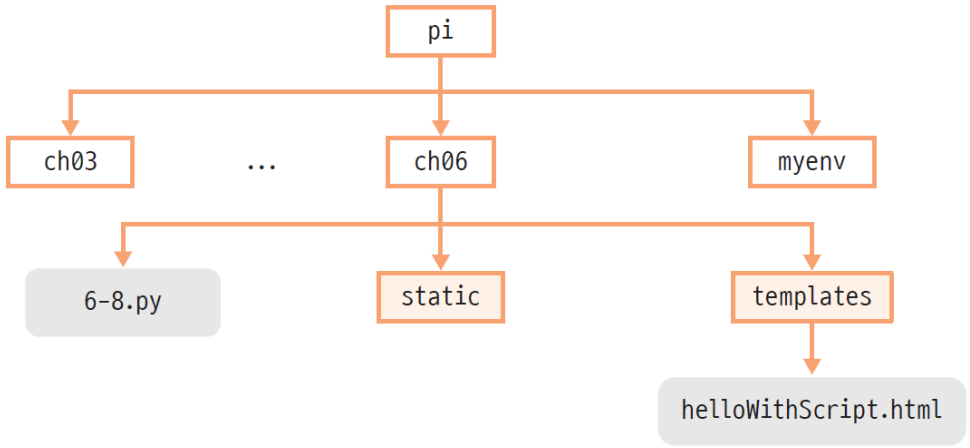
```
render_template('helloWithScript.html', name='jmlee', dept='Mobile Software')
```

예제 6–8 스크립트를 가진 템플릿 HTML과 render_template() 활용

이 예제를 통해 스크립트를 가진 템플릿 HTML, helloWithScript.html을 작성하고, render_template ( ) 함수로 렌더링하여 동적 콘텐츠를 생성하는 사례를 알아본다. 이 예제를 위한 디렉터리 구조는 다음과 같다.

helloWithScript.html

```
<!doctype html>
<html>
<head>
   <title>Script 사용하기</title>
</head>
<body>
     <h2>Script 사용하기</h2>
 <hr>
 <h2>Hello,
   {% if(name == 'jmlee'): %}
   {% print('Jae Moon Lee!') %}
   {% elif(name == 'kitae'): %}
   {% print('Ki Tae Hwang!') %}
   {% else: %}
   {% print('Unknown') %}
   {% endif %}
 </h2>
 <h2>당신의 소속은 {% print(dept) %}입니다.</h2>
</body>
</html>
```




```
<h2>Hello,
Jae Moon Lee!
</h2>
<h2>당신의 소속은 Mobile Software입니다.</h2>
```

예제 6–8 (계속)

6-8.py

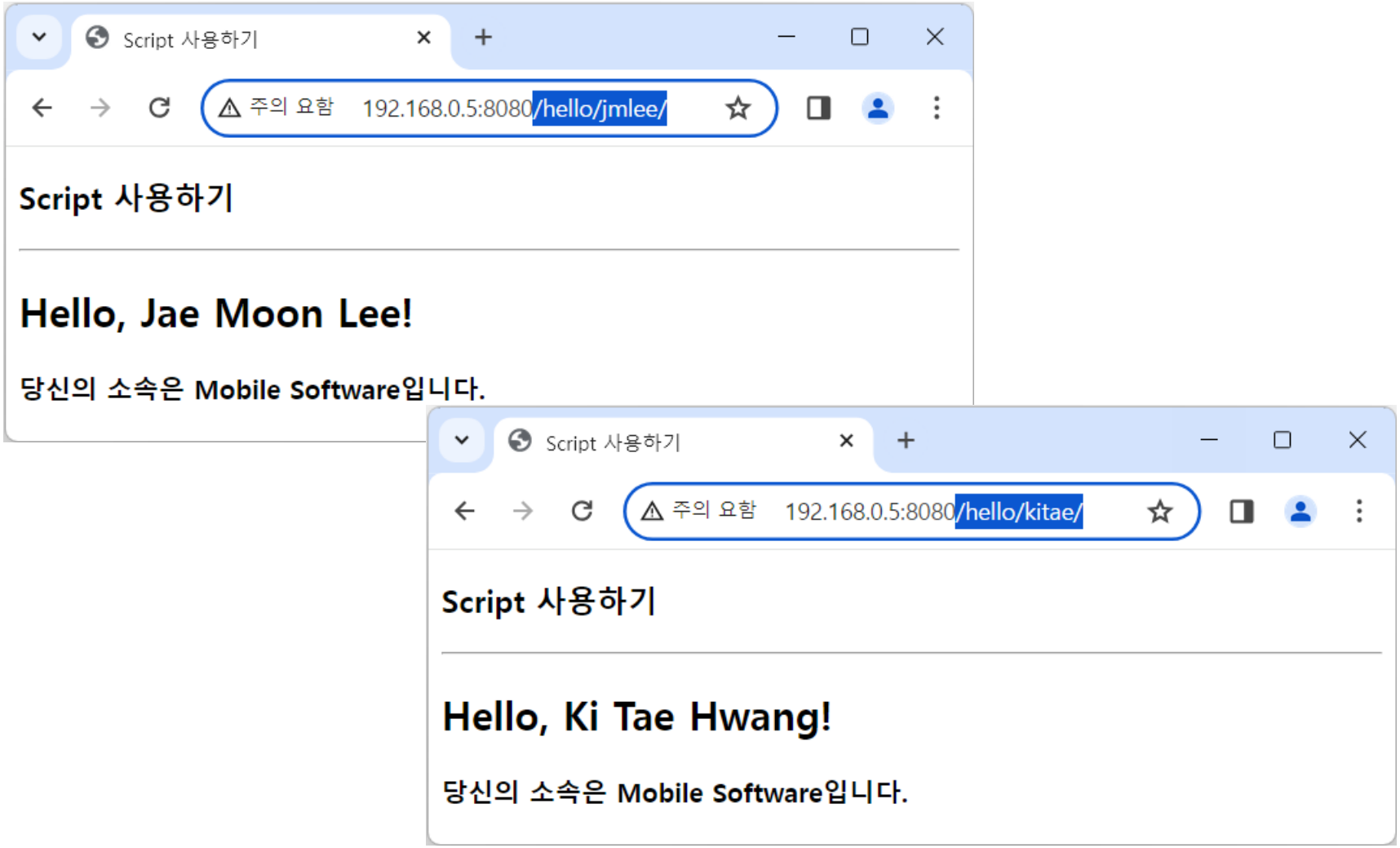
```
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/hello/<myname>/') # myname은 variable
def hello(myname):
    return render_template('helloWithScript.html', name=myname, dept='Mobile Software')

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

```
(myenv) pi@pi:~/ch06 $ python 6-8.py
* Serving Flask app "6-8" (lazy loading)
...(생략)
* Debugger PIN: 235-625-251
192.168.0.50 - - [16/May/2023 00:34:54] "GET /hello/jmlee/ HTTP/1.1" 200 -
192.168.0.50 - - [16/May/2023 00:37:23] "GET /hello/kitae/ HTTP/1.1" 200 -
```

예제 6-8 (계속)


Script 사용하기
주의 요함
192.168.0.5:8080/hello/jmlee/
Script 사용하기
Hello, Jae Moon Lee!
당신의 소속은 Mobile Software입니다.
Script 사용하기
주의 요함
192.168.0.5:8080/hello/kitae/
Script 사용하기
Hello, Ki Tae Hwang!
당신의 소속은 Mobile Software입니다.

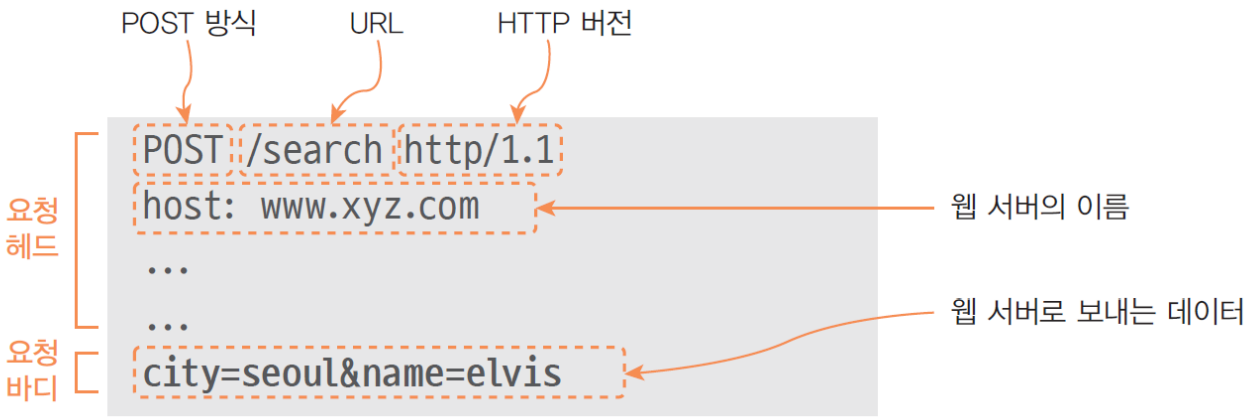
6.5 웹브라우저와 플라스크 사이의 데이터 전송

GET과 POST

- 웹 데이터 전송 프로토콜 중 대표적인 2가지
  - GET 방식 – 데이터를 헤드 부분에 포함
  - POST 방식 - 데이터를 바디에 포함
- GET 방식



- POST 방식



플라스크의 request 객체

- request 객체
  - 웹 브라우저로부터 받은 HTTP 요청 패킷을 분석
  - 요청 내용을 request라는 이름의 객체에 저장
  - 웹 서버 응용프로그램이 활용할 수 있도록 제공

request 객체의 속성	내용
request.url	웹 브라우저에서 사용자가 입력한 전체 텍스트 (http://192.168.0.5:8080/search?city=seoul&name=elvis)
request.full_path	URL 텍스트 이하의 모든 데이터 (/search?city=seoul&name=elvis)
request.path	URL 텍스트 데이터만(/search)
request.method	"GET"이나 "POST" 문자열
request.args	GET 방식에서 데이터 부분의 [이름, 값] 쌍의 딕셔너리
request.form	POST 방식에서 데이터 부분의 [이름, 값] 쌍의 딕셔너리
request.values	args와 form 속에 저장된 [이름, 값]을 모두 저장

  - request 객체 사용

```
from flask import request
```

예제 6-9 GET 방식의 request 객체의 속성 검사

request 객체를 활용하여 웹 브라우저에서 보내온 요청 패킷의 정보를 출력하는 프로그램을 작성하라.

6-9.py

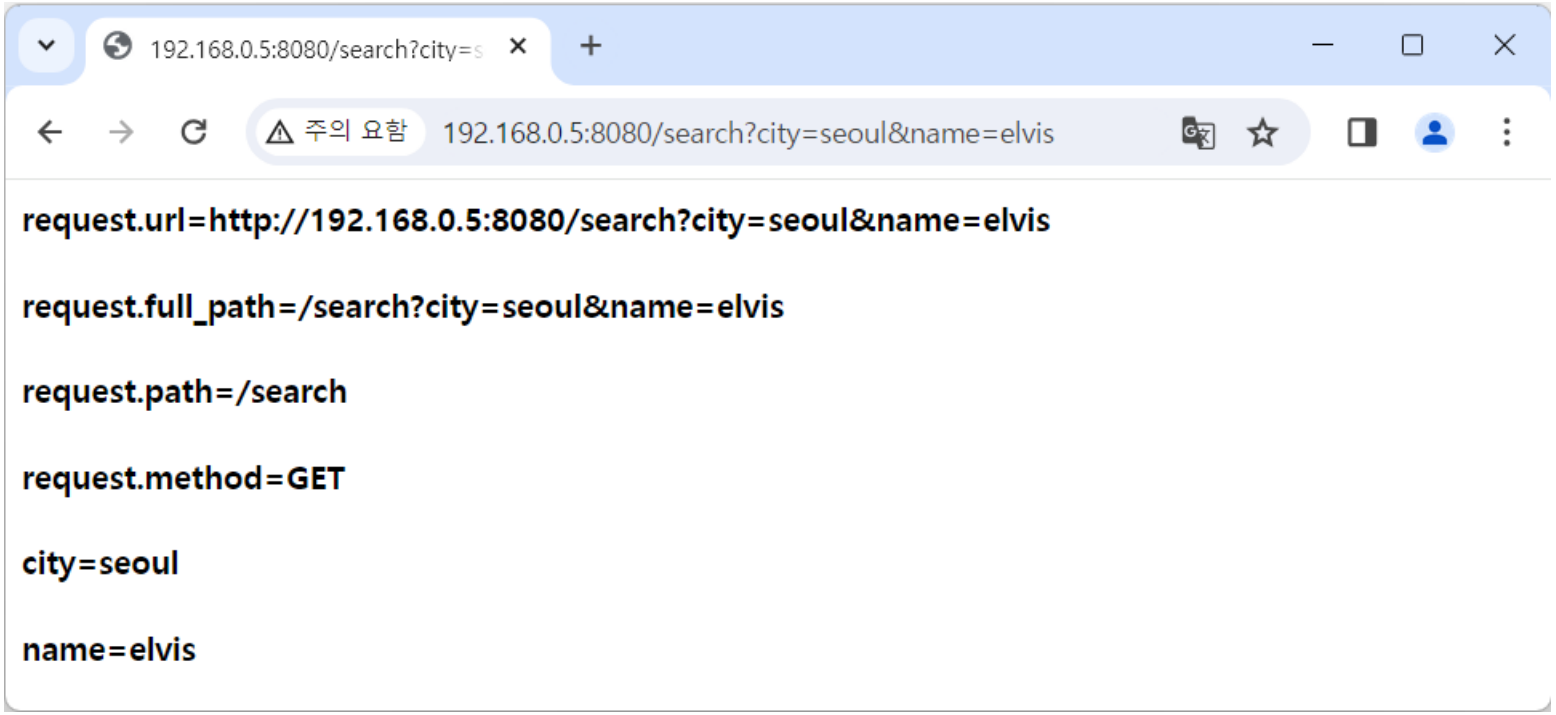
```
from flask import Flask, request
app = Flask(__name__)

@app.route('/search')
def echo_request( ):
    str = "<h4>request.url=" + request.url + "</h4>"
    str += "<h4>request.full_path=" + request.full_path + "</h4>"
    str += "<h4>request.path=" + request.path + "</h4>"
    str += "<h4>request.method=" + request.method + "</h4>"
    if request.method == 'GET':
        str += "<h4>city=" + request.args['city'] + "</h4>"
        str += "<h4>name=" + request.args['name'] + "</h4>"
    elif request.method == 'POST':
        str += "<h4>city=" + request.form['city'] + "</h4>"
        str += "<h4>name=" + request.form['name'] + "</h4>"
        str += "</h4>"
    return str;

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

예제 6-9 (계속)


```
(myenv) pi@pi:~/ch06 $ python 6-9.py
* Serving Flask app "6-9" (lazy loading)
...(생략)
* Debugger PIN: 235-625-251
192.168.0.50 - - [15/May/2023 15:40:51] "GET /search?city=seoul&name=elvis HTTP/1.1" 200 -
```



웹 브라우저로부터 GET 방식의 데이터 수신

- 웹 브라우저에 직접 입력

```
IP:8080/.../?name1=value1&name2=value2&...&name3=valuen
```

예제 6-10 GET 방식으로 사칙 연산하기

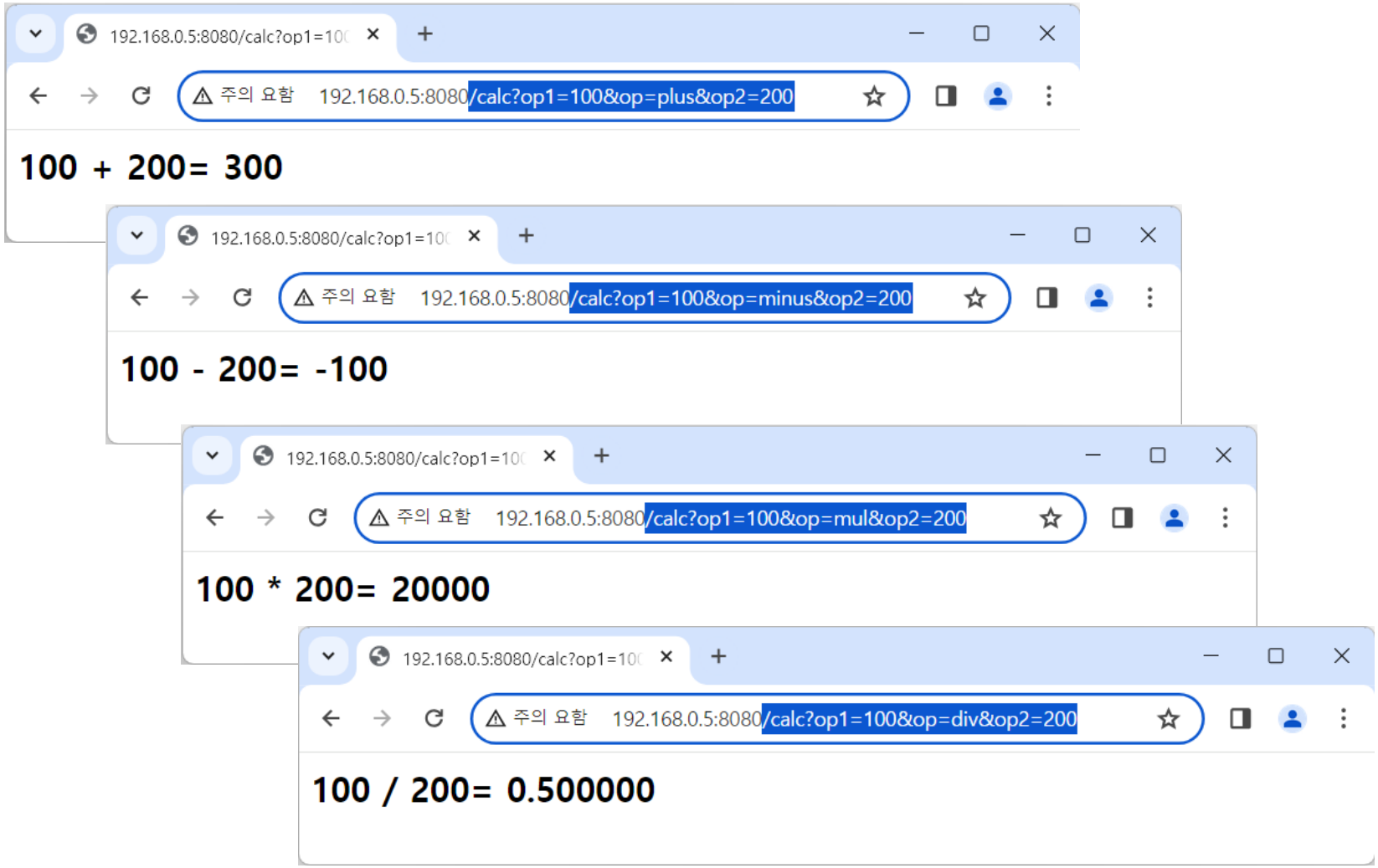
다음 사례와 같은 URL을 이용하여 사칙 연산을 수행하는 웹 서버 프로그램을 작성하라.

- /calc?op1=100&op=plus&op2=200 - > 100 + 200를 수행하고 300를 응답으로 리턴
- /calc?op1=100&op=minus&op2=200 - > 100 - 200를 수행하고 - 100를 응답으로 리턴
- /calc?op1=100&op=mul&op2=200 - > 100 * 200를 수행하고 20000을 응답으로 리턴
- /calc?op1=100&op=div&op2=200 - > 100 / 200을 수행하고 0.500000을 응답으로 리턴

6-10.py

```
from flask import Flask, request
app = Flask(__name__)
@app.route('/calc')
def calc( ):
    if request.method != 'GET':
        return ""
    op1 = request.args['op1'] # 'op1' 키에 대한 값 데이터
    op2 = request.args['op2'] # 'op2' 키에 대한 값 데이터
    op = request.args['op'] # 'op' 키에 대한 값(연산자) 데이터
    num1 = int(op1) # 정수로 변환
    num2 = int(op2) # 정수로 변환
    if op == 'plus':
        return "<h2>%d + %d= %d</h2>" % (num1, num2, num1+num2)
    elif op == 'minus':
        return "<h2>%d - %d= %d</h2>" % (num1, num2, num1-num2)
    elif op == 'mul':
        return "<h2>%d * %d= %d</h2>" % (num1, num2, num1*num2)
    elif op == 'div':
        if num2 == 0:
            return "<h2>0으로 나눌 수 없습니다.</h2>"
        return "<h2>%d / %d= %f</h2>" % (num1, num2, num1/num2)
    else:
        return "<h2>잘못된 연산자입니다.</h2>"
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

예제 6-10 (계속)

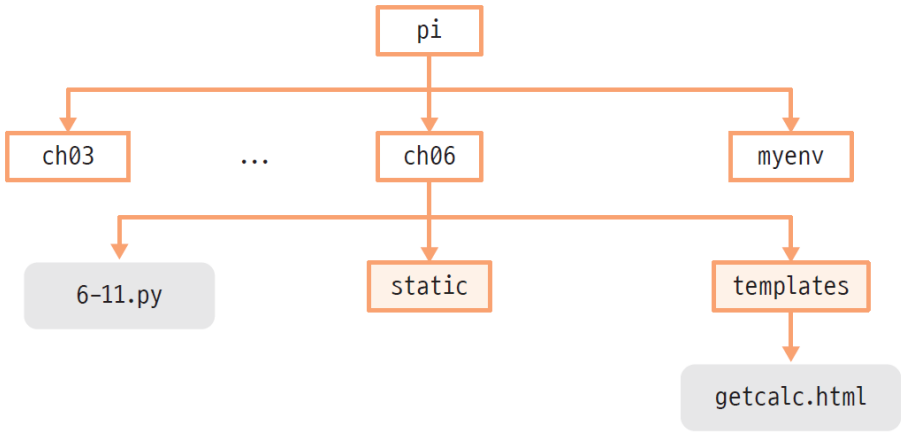

192.168.0.5:8080/calc?op1=100&op=plus&op2=200
100 + 200= 300
192.168.0.5:8080/calc?op1=100&op=minus&op2=200
100 - 200= -100
192.168.0.5:8080/calc?op1=100&op=mul&op2=200
100 * 200= 20000
192.168.0.5:8080/calc?op1=100&op=div&op2=200
100 / 200= 0.500000

예제 6-11 폼 태그(<form>)를 이용하여 GET 방식으로 웹 서버에 데이터 전송

예제 6-10은 사용자가 웹 브라우저에서 직접 URL을 입력하여 요청하는 사례였다. 이 예제에서는 <form>태그를 이용하여 더욱 자연스럽게, 웹 페이지에서 웹 서버로 요청을 보내는 형태로 만들어본다. 이 예제를 위한 웹 서버의 디렉터리 구조는 다음과 같다.

getcalc.html

```
<!doctype html>
<html>
   <head>
    <title>GET 방식으로 데이터 전송</title>
  </head>
<body>
   <h3>GET 방식으로 사칙연산 처리</h3>
   <hr>
   <form action="/calc" method="get">
      <input name="op1" value="100" type="text" />
      <input name="op" value="plus" type="text" />
      <input name="op2" value="200" type="text" />
      <input value="계산" type="submit" />
   </form>
</body>
</html>
```





예제 6-11 (계속)

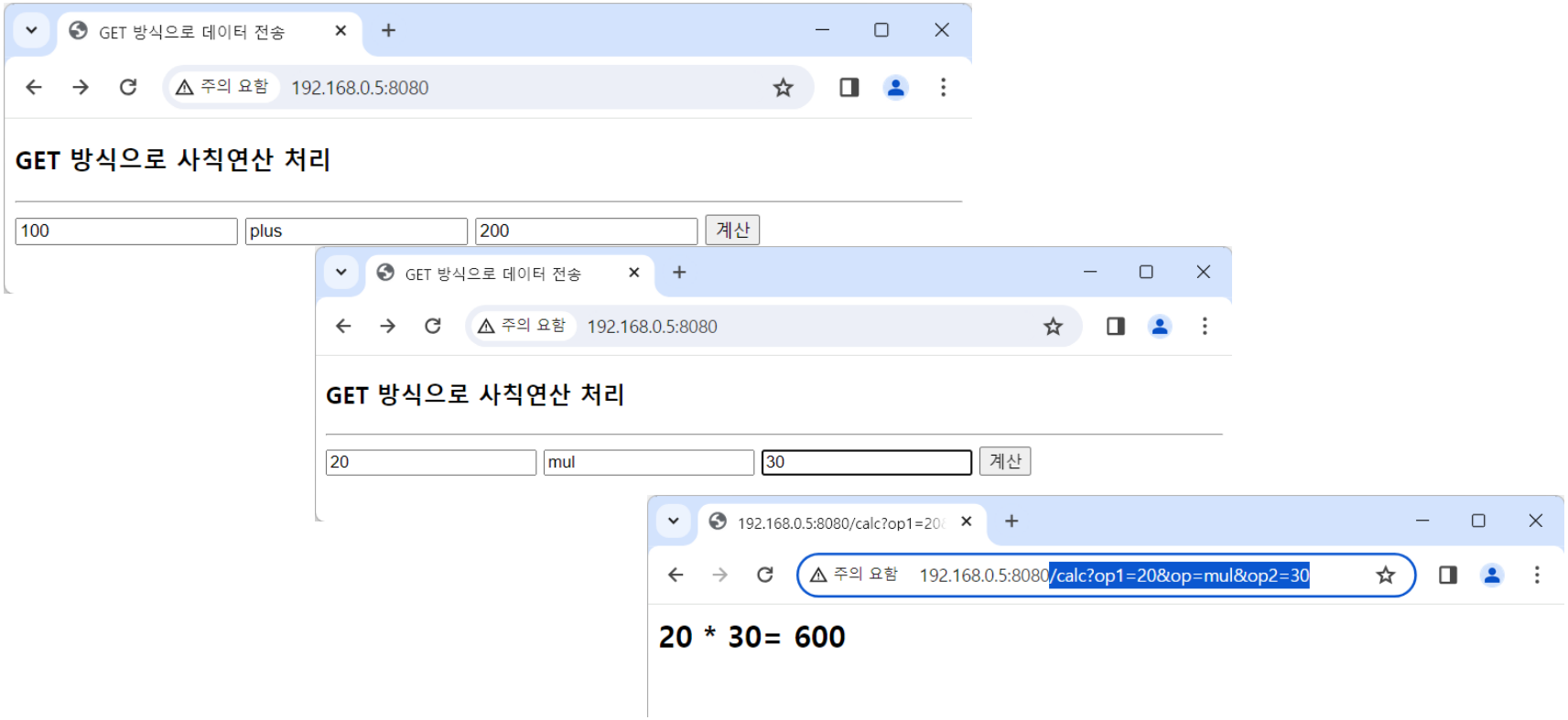
웹 브라우저는 이(getcalc.html)를 받아 출력하고 사용자 입력을 받은 후 <form> 태그의 action 속성에 지정된 문자열을 URL로 하고 그 뒤에 ?를 붙인다. 그리고 <form> 태그 내 모든 <input> 태그의 name 속성과 value 속성을 name=value&..의 모양으로 연결하여 ? 문자 뒤에 덧붙인다. 그런 후 GET 방식으로 웹 서버에 전송한다.

6-11.py

```
from flask import Flask, request, render_template
app = Flask(__name__)
@app.route('/')
def init( ):
    return render_template("getcalc.html")
@app.route('/calc')
def calc( ):
    if request.method != 'GET':
        return ""
    op1 = request.args['op1'] # 'op1' 키에 대한 값 데이터
    op2 = request.args['op2'] # 'op2' 키에 대한 값 데이터
    op = request.args['op'] # 'op' 키에 대한 값(연산자) 데이터
    num1 = int(op1) # 정수로 변환
    num2 = int(op2) # 정수로 변환
    if op == 'plus':
        return "<h2>%d + %d= %d</h2>" % (num1, num2, num1+num2)
    elif op == 'minus':
        return "<h2>%d - %d= %d</h2>" % (num1, num2, num1-num2)
    elif op == 'mul':
        return "<h2>%d * %d= %d</h2>" % (num1, num2, num1*num2)
    elif op == 'div':
        if num2 == 0:
            return "<h2>0으로 나눌 수 없습니다.</h2>"
        return "<h2>%d / %d= %f</h2>" % (num1, num2, num1/num2)
    else:
        return "<h2>잘못된 연산자입니다.</h2>"
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

예제 6-11 (계속)


```
(myenv) pi@pi:~/ch06 $ python 6-11.py
* Serving Flask app "6-11" (lazy loading)
...(생략)
* Debugger PIN: 156-042-313
192.168.0.50 - - [19/May/2023 10:26:23] "GET / HTTP/1.1" 200 -
192.168.0.50 - - [19/May/2023 10:26:59] "GET /calc?op1=20&op=mul&op2=30 HTTP/1.1" 200 -
```



POST 방식으로 데이터 전송

- 전송할 데이터는 URL에 직접 입력할 수 없음

```
<form action="/calc" method="post">
  ...
  <input value="계산" type="submit" />
</form>
```

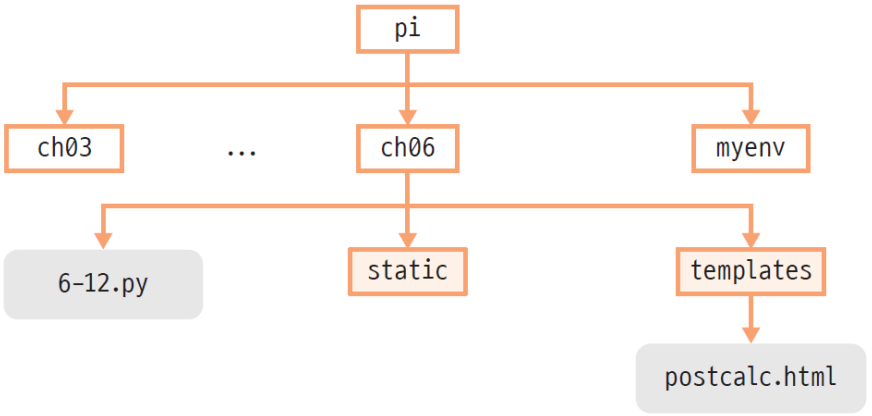
- 웹 서버로 전송할 데이터
  - 요청 패킷의 바디 부분에 전달
  - 데이터의 구성은 name=value&name=value... 모양, GET 방식과 동일
- 플라스크는 수신 패킷의 바디 부분 분석
  - request.form에 {'num1': '100, 'op': 'plus', 'num2': '200'} 모양으로 저장

예제 6-12 폼 태그(<form>)를 이용하여 POST 방식으로 웹 서버에 데이터 전송

이 예제에서는 예제 6-11의 상황을 POST 방식으로 구현한다. 예제를 실행하기 위한 웹 서버의 구성은 다음과 같다.

postcalc.html

```
<!doctype html>
<html>
   <head>
      <title>POST 방식으로 데이터 전송</title>
   </head>
<body>
   <h3>POST 방식으로 사칙연산 처리</h3>
   <hr>
   <form action="/calc" method="post">
      <input name="op1" value="100" type="text" />
      <input name="op" value="plus" type="text" />
      <input name="op2" value="200" type="text" />
      <input value="계산" type="submit" />
   </form>
</body>
</html>
```





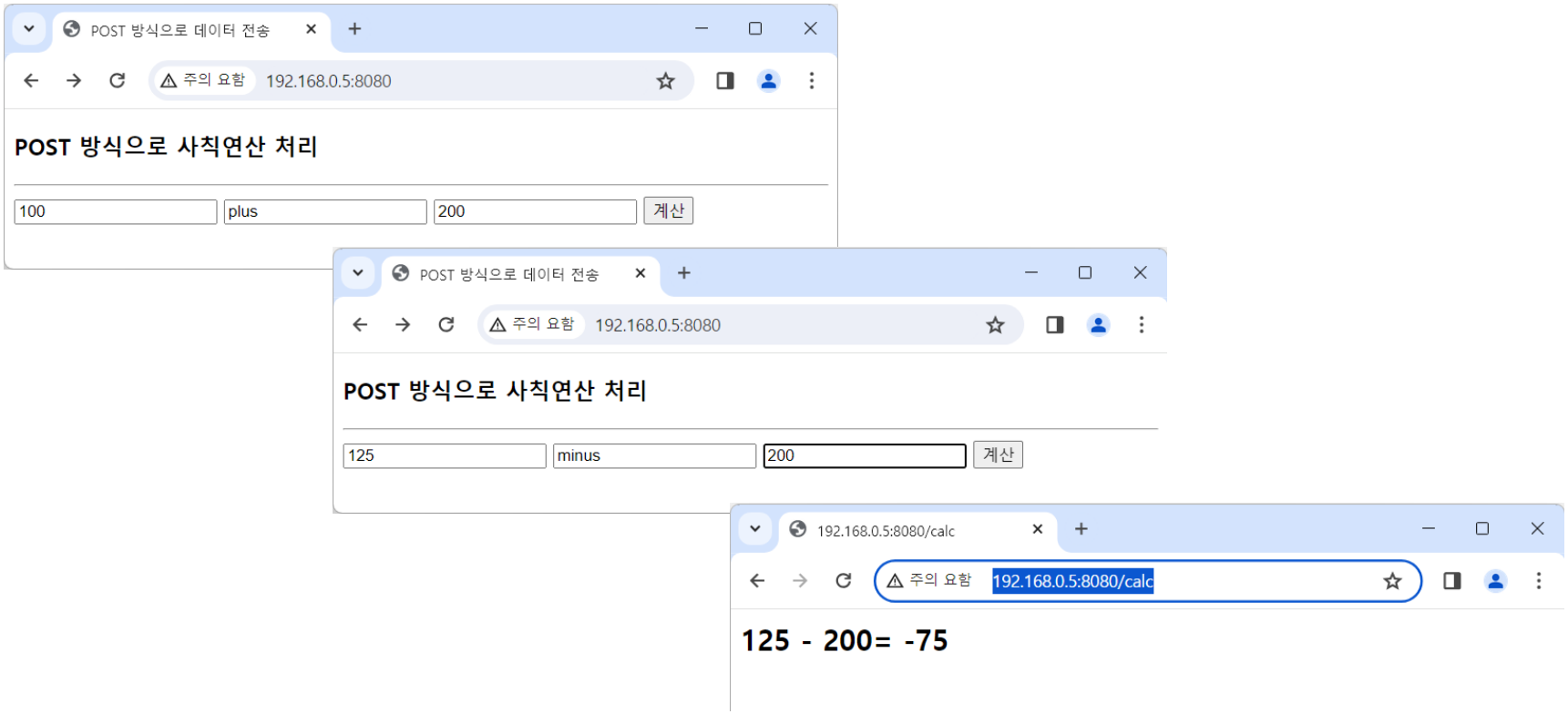
예제 6-12 (계속)

6-12.py

```
from flask import Flask, request, render_template
app = Flask(__name__)
@app.route('/')
def init( ):
    return render_template("postcalc.html")
@app.route('/calc', methods=['POST']) # method가 POST 인 경우만 라우팅
def calc( ):
    op1 = request.form['op1'] # 'op1' 키에 대한 데이터
    op2 = request.form['op2'] # 'op2' 키에 대한 데이터
    op = request.form['op'] # 'op' 키에 대한 데이터
    num1 = int(op1) # 정수로 변환
    num2 = int(op2) # 정수로 변환
    if op == 'plus':
        return "<h2>%d + %d= %d</h2>" % (num1, num2, num1+num2)
    elif op == 'minus':
        return "<h2>%d - %d= %d</h2>" % (num1, num2, num1-num2)
    elif op == 'mul':
        return "<h2>%d * %d= %d</h2>" % (num1, num2, num1*num2)
    elif op == 'div':
        if num2 == 0:
            return "<h2>0으로 나눌 수 없습니다.</h2>"
        return "<h2>%d / %d= %f</h2>" % (num1, num2, num1/num2)
    else:
        return "<h2>잘못된 연산자입니다.</h2>"
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

예제 6-12 (계속)


```
(myenv) pi@pi:~/ch06 $ python 6-12.py
* Serving Flask app "6-12" (lazy loading)
...(생략)
* Debugger PIN: 156-042-313
192.168.0.50 - - [19/May/2023 11:52:34] "GET / HTTP/1.1" 200 -
192.168.0.50 - - [19/May/2023 11:52:36] "POST /calc HTTP/1.1" 200 -
```

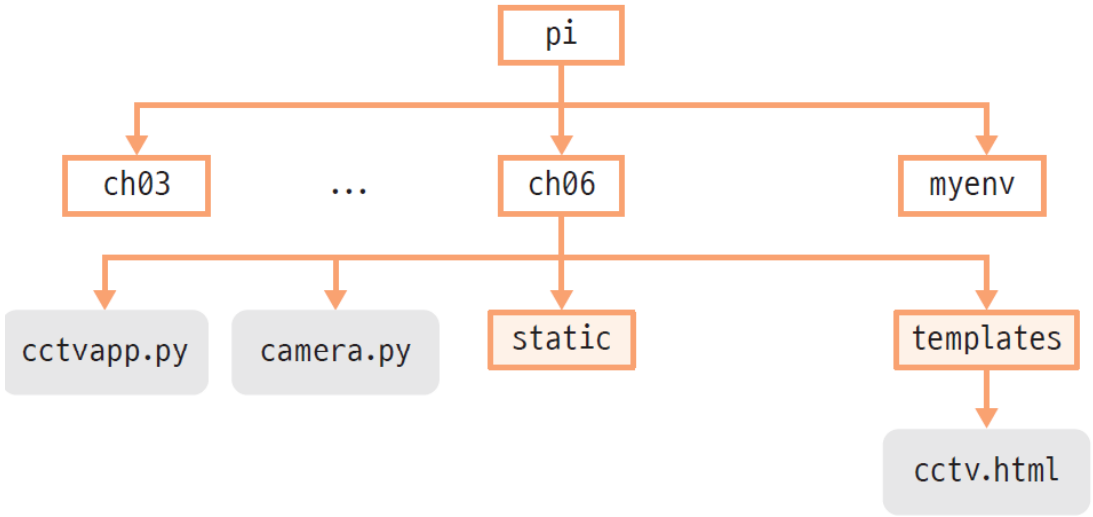


66

6.6 플라스크 종합 응용 – 원격 카메라 촬영 및 CCTV 웹 서버 만들기

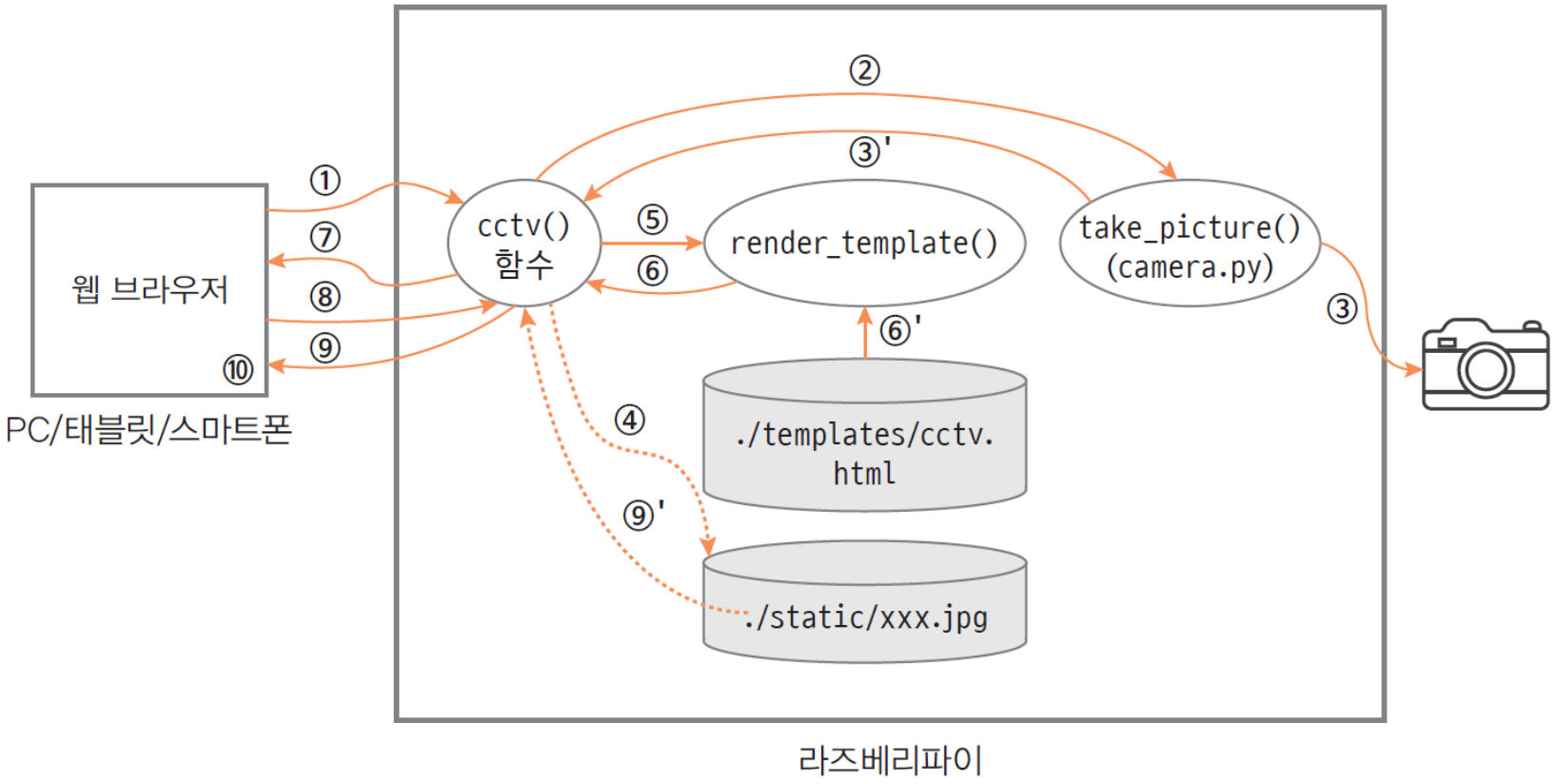
프로그램 구조

- 웹서버 구조



  - camera.py
    - 사진을 촬영하는 파이썬 코드 부분
  - cctvapp.py
    - 플라스크를 활용한 웹 서버 응용프로그램. camera.py를 임포트하여 사진 촬영
  - cctv.html
    - 템플릿 HTML 파일

프로그램 동작


②
③'
①
⑦
⑧
⑨
⑩
웹 브라우저
PC/태블릿/스마트폰
cctv()
함수
⑤
⑥
render_template()
take_picture()
(camera.py)
③
⑥'
④
⑨'
./templates/cctv.
html
./static/xxx.jpg
라즈베리파이

프로그램 동작 과정

- (1) 웹 브라우저에서 라즈베리파이의 8080 포트에 접속하고 사진 촬영을 요청하는 URL 전송
- (2) cctvapp.py의 cctv() 함수가 실행되고, cctv() 함수는 카메라 사진을 촬영하기 위해 camera.py에 작성된 take_picture() 함수를 호출하여 카메라로 사진 촬영
- (3) take_picture() 함수는 사진 촬영 후 사진의 이미지 데이터 리턴
- (4) cctv() 함수는 리턴받은 이미지 데이터를 ./static 디렉터리에 jpg 파일로 저장. jpg 파일의 이름은 현재 시간 값으로 붙임
- (5) cctv() 함수는 cctv.jpg 사진 파일의 이름을 매개변수로 하여 플라스크의 render_template() 함수 호출
- (6) render_template() 함수는 템플릿 파일 cctv.html 내 스크립를 실행, <img> 태그의 src 속성에 cctv.jpg 파일 이름을 지정한 후, 수정된 cctv.html 파일의 텍스트를 문자열 형태로 리턴
- (7) ccvt() 함수는, render_template() 함수가 리턴한 텍스트를 웹 브라우저로 전송
- (8) 웹 브라우저는 수신한 HTML 텍스트 해석. HTML 텍스트에 포함된 <img> 태그를 발견하고, <img>의 src 속성에 지정된 cctv.jpg 파일을 수신하기 위해, 다시 jpg 파일을 웹 서버에 요청
- (9) 다시 cctvapp.py의 cctv() 함수가 실행되어, 요청받은 jpg 파일을 읽어 전송
- (10) 웹 브라우저는 이미지와 함께 HTML 텍스트 출력

camara.py

- 다음 3개의 함수로 구성
  - init() - OpevCV의 VideoCapture 객체 초기화
  - take_picture() - 사진을 촬영하고 이미지 데이터 리턴
  - final() - 카메라 사용이 끝났을 때 VideoCapture 객체 해제

camera.py

```
import cv2
camera = None
def init(camera_id=0, width=640, height=480, buffer_size=1):
    global camera
    camera = cv2.VideoCapture(camera_id, cv2.CAP_V4L)
    camera.set(cv2.CAP_PROP_FRAME_WIDTH, width)
    camera.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
    camera.set(cv2.CAP_PROP_BUFFERSIZE, buffer_size)
def take_picture(most_recent=False):
    global camera
    # most_recent가 True이면 버퍼에 저장되어 있는 프레임을 전부 버리도록 한다.
    len = 0 if most_recent == False else camera.get(cv2.CAP_PROP_BUFFERSIZE)
    while(len > 0):
        camera.grab( ) # 버퍼에 저장되어 있는 프레임을 버린다.
        len -= 1
        success, image = camera.read( )
        if not success:
            return None
        return image
def final( ):
    if camera != None:
        camera.release( )
    camera = None
```

cctvapp.py

- cctvapp.py
  - 카메라 객체 생성
  - 브라우저로부터 요청이 있으면 카메라 객체를 이용하여 이미지를 캡쳐하여 파일에 저장
  - 이미지에 대한 파일 이름을 브라우저에 전송

cctvapp.py

```
import cv2
from flask import Flask, render_template
import camera # 카메라 모듈, camera.py 임포트

app = Flask(__name__) # 플라스크 객체 생성
camera.init(width=640, height=480) # 카메라 모듈 초기화

@app.route('/cctv')
def cctv( ):
    image = camera.take_picture(most_recent=True) # 사진 촬영. 이미지 읽기
    if image is not None:
        cv2.imwrite('./static/cctv.jpg', image) # 이미지를 ./static 디렉터리에 파일로 저장
        # file_name를 사용하여 cctv.html를 변경하고, 파일 텍스트를 문자열로 리턴
        return render_template("cctv.html", fname='/static/cctv.jpg')

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080) # debug=True 속성을 사용하면 오류 발생
```

템플릿 HTML 파일 - cctv.html

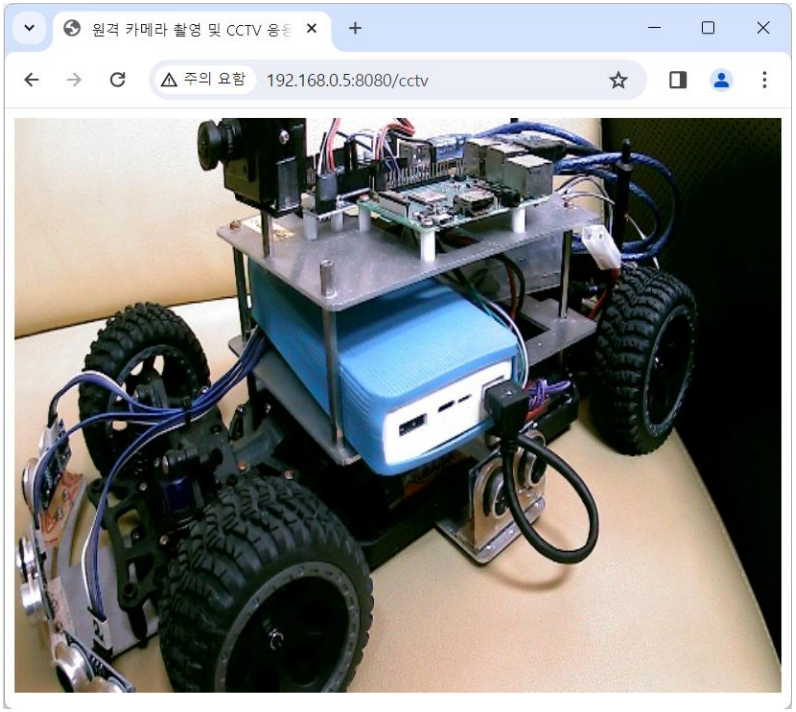
- cctv.html
  - render_template() 함수에서 렌더링하는 cctv.html 파일은 이미지 한 개를 출력하도록 <img> 태그 포함
  - cctv.html

```
<!doctype html>
<html>
  <head><title>원격 카메라 촬영 및 CCTV 응용</title></head>
  <body>
    <img src="{{fname}}" width="640" height="480"/>
  </body>
</html>
```

실행

실행 및 접속

```
(myenv) pi@pi:~/ch06 $ python cctvapp.py
* Serving Flask app "cctvapp" (lazy loading)
...(생략)
* Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
192.168.0.50 - - [19/May/2023 11:55:34] "GET /cctv HTTP/1.1" 200 -
192.168.0.50 - - [19/May/2023 11:52:35] "GET /static/cctv.jpg HTTP/1.1" 200 -
```



CCTV 스트리밍

- CCTV 스트리밍
  - 웹 브라우저에서 자동으로 '새로 고침'이 반복적으로 진행되면, CCTV 스트리밍과 동일한 효과
  - meta 태그를 이용한 반복 '새로 고침'

```
<meta http-equiv="refresh" content="0.5;url={%print(request.path)%}" >
```

  - cctv.html 수정

```
<!doctype html>
<html>
   <head>
      <title>원격 카메라 촬영 및 CCTV 응용</title>
      <meta http-equiv="refresh" content="0.5;url={%print(request.path)%}" >
   </head>
   <body>
      <img src="{{fname}}" width="640" height="480"/>
   </body>
</html>
```

CCTV 스트리밍

실행 및 접속

```
(myenv) pi@pi:~/ch06 $ python cctvapp.py
* Serving Flask app "cctvapp" (lazy loading)
...(생략)
* Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
192.168.0.50 - - [23/May/2023 16:47:56] "GET /cctv HTTP/1.1" 200 -
192.168.0.50 - - [23/May/2023 16:47:56] "GET /static/16848280759.jpg HTTP/1.1" 200 -
192.168.0.50 - - [23/May/2023 16:47:56] "GET /cctv HTTP/1.1" 200 -
192.168.0.50 - - [23/May/2023 16:47:56] "GET /static/16848280767.jpg HTTP/1.1" 200 -
192.168.0.50 - - [23/May/2023 16:47:57] "GET /cctv HTTP/1.1" 200 -
192.168.0.50 - - [23/May/2023 16:47:57] "GET /static/16848280773.jpg HTTP/1.1" 200 -
192.168.0.50 - - [23/May/2023 16:47:58] "GET /cctv HTTP/1.1" 200 -
192.168.0.50 - - [23/May/2023 16:47:58] "GET /static/16848280780.jpg HTTP/1.1" 200 -
....[반복된다]
```

